

Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review

Mohammad Amin Zadenoori^{1a}, Jacek Dąbrowski^{2b}, Waad Alhoshan^{3c},
Liping Zhao^{4d}, Alessio Ferrari^{5e}

^a*Department of Statistics, University of Padova, Via Cesare Battisti
241, Padova, 35121, Italy*

^b*Lero, the Research Ireland Centre for Software, University of Limerick, Limerick, V94
T9PX, Ireland*

^c*Imam Mohammad Ibn Saud Islamic University (IMSIU), Riyadh, 13317, Saudi Arabia*

^d*University of Manchester, Oxford Road, Manchester, M13 9PL, United Kingdom*

^e*University College Dublin (UCD), Belfield, Dublin 4, Dublin, D04 V1W8, Ireland*

Abstract

Large Language Models (LLMs) are finding applications in numerous domains, and Requirements Engineering (RE) is increasingly benefiting from their capabilities to assist with complex, language-intensive tasks. This paper presents a systematic literature review of 74 primary studies published between 2023 and 2024, examining how LLMs are being applied in RE. The study categorizes the literature according to several dimensions, including publication trends, supported RE activities, prompting strategies, and evaluation methods. Our findings indicate notable patterns, among which we observe substantial differences compared to previous works leveraging standard Natural Language Processing (NLP) techniques. Most of the studies focus on using LLMs for requirements elicitation and validation, rather than defect detection and classification, which were dominant in the past. Researchers have also broadened their focus and addressed novel tasks, e.g., test generation, exploring the integration of RE with other software engineering (SE)

¹amin.zadenoori@unipd.it

²jacek.dabrowski@lero.ie

³waboud@imamu.edu.sa

⁴liping.zhao@manchester.ac.uk

⁵alessio.ferrari@ucd.ie

disciplines. Although requirements specifications remain the primary focus, other artifacts are increasingly considered, including issues from issue tracking systems, regulations, and technical manuals. The studies mostly rely on GPT-based models, and often use Zero-shot or Few-shot prompting. They are usually evaluated in controlled environments, with limited use in industry settings and limited integration in complex workflows. Our study outlines important future directions, such as leveraging the potential to expand the influence of RE in SE, exploring less-studied tasks, improving prompting methods, and testing in real-world environments, considering the usefulness of these tools, besides their performance. Our contribution also helps researchers and practitioners use LLMs more effectively in RE, by providing a list of identified tools leveraging LLMs for RE, as well as datasets.

Keywords: Requirements engineering, Large language models, Prompt engineering, Systematic literature review.

1. Introduction

The central role of Natural Language Processing (NLP) in Requirements Engineering (RE) has been consistently studied [1, 2]. Early observations, such as Abbott and Moorhead’s [3] assertion that “natural language is the most suitable language for requirements”, laid the groundwork for its adoption in RE. However, traditional NLP approaches—those preceding the advent of deep learning, such as rule-based systems and statistical methods—often struggled to handle the nuances, ambiguities, and complexities inherent in real-world requirements documents. These limitations stemmed from their reliance on predefined rules or shallow statistical models, which lacked the ability to deeply contextualize language.

The introduction of deep learning marked a shift in NLP capabilities, enabling more robust handling of linguistic intricacies through neural network architectures and word embeddings [4]. This evolution progressed further with the development of transformer-based models, exemplified by BERT (Bidirectional Encoder Representations from Transformers) [5], which introduced bidirectional context understanding, enhancing tasks like requirements classification [6] and traceability [7] in RE. Generative Large Language Models (LLMs) made another step forward. These models are designed to create new content, such as text, by predicting and generating sequences of words based on vast training data. Prominent examples of LLMs include OpenAI’s

GPT models (e.g., ChatGPT [8]), Meta’s LLaMA [9], and DeepSeek-R1 [10]. While BERT, a bidirectional-only encoder, excels at understanding context for tasks like classification or question-answering, generative LLMs use autoregressive or similar architectures to produce coherent, contextually relevant outputs, often for creative writing, dialogue, or code generation. Their core difference lies in their purpose: BERT analyzes and interprets, while generative LLMs synthesize and create, leveraging patterns learned from diverse datasets to mimic human-like text production [11]. Powered by Deep learning and expansive transformer architectures, these LLMs offer transformative potential for RE, and their use in RE research has quickly become widespread.

This Systematic Literature Review (SLR) investigates the research landscape of LLMs for RE (LLM4RE), i.e., the use of LLM to support RE-relevant activities. We focus our study on generative pre-trained transformers—commonly referred to as LLMs and their successors [12]—rather than encoder-only transformers, such as BERT. The reason for this choice is that we want to focus on models whose capabilities extend beyond text representation and classification, and encompass advanced abilities in text understanding and production. Furthermore, the introduction of LLMs has marked a paradigm shift in everyday usage of NLP techniques, and we want to see how the RE field is being shaped by these revolutionary tools.

In this work, we identified 74 primary studies published mostly in conferences (66%) and workshops (23%), reflecting a collaborative, early-stage research landscape. The analysis focuses on core RE tasks such as *Requirements Elicitation* and *Validation*. Prompt engineering strategies—particularly *Zero-shot* (44%) and *Few-shot* (29%) prompting—dominate the field, while emerging techniques like *RAG* (6%) and *Interactive prompting* (5%) remain underexplored. *GPT-based models* lead adoption (90%), accompanied by a smaller but growing interest in alternatives such as *LLaMA* (15%). Many studies (61%) share their source code and prompts, indicating a move toward open science, but shared datasets are still scarce. Evaluation still relies heavily on *Laboratory Experiments* (75%), which highlights the need for real-world validations and field studies. Underexplored areas (e.g., *Requirements Retrieval*, *Terminology Extraction*), more diverse models, advanced prompting methods, as well as deeper practitioner collaboration, present key opportunities for further advancing LLM4RE, and broaden its impact on the Software Engineering (SE) field and beyond.

The remainder of the paper is structured as follows: Section 2 presents

the concepts and definitions. Section 3 discusses related works. In Section 4, we describe the research methodology. Section 5 covers the results, while Section 6 provides a summary and discussion. Section 7 outlines the threats to validity, and the final section presents the conclusions.

Data Availability. The spreadsheets resulting from our data extraction and grouping, along with all synthesized data, including generated statistics and classifications, are publicly available in the supplementary material of this survey [13].

Disclaimer. This paper is a preliminary study. No systematic cross-checking has been performed on the retrieved and analysed studies. The paper will be updated, and future versions will include appropriate measures to increase the reliability of the findings.

2. Background

This section introduces key concepts associated with RE and LLMs. Given these premises, we then outline the different dimensions investigated in our study, which are the foundations of the classification scheme used in this paper to categorise the research contributions.

2.1. Requirement Engineering

Requirements Engineering (RE) is the subfield of Software Engineering (SE) concerned with the systematic processes of eliciting, documenting, analyzing, verifying, validating, and managing system and software requirements throughout the entire system lifecycle [14]. RE activities are predominantly conducted in natural language (NL), which serves as the most widely understood medium of communication. Using NL facilitates effective knowledge exchange among the diverse stakeholders involved in system development, including customers, developers, business analysts, and assessors.

RE is structured around a series of interrelated phases, each addressing specific aspects of requirement development and management. The first phase, *Elicitation*, focuses on understanding the goals, objectives, and needs of stakeholders for building a proposed software system. This phase involves gathering information from customers, users, and other stakeholders through interviews, workshops, surveys, and observation, ensuring that the system’s intended functionality and qualities are captured comprehensively. Following elicitation, *Analysis* evaluates the quality of the recorded requirements

and identifies potential anomalies, such as ambiguity, inconsistency, or incompleteness. This step ensures that requirements are clear, correct, and feasible before they are used in subsequent activities. Once requirements have been analyzed, the *Modeling* phase translates them into interpretable and structured conceptual representations. These models—ranging from UML diagrams to ontologies—support both technical and non-technical stakeholders in understanding system functionality. The *Management* phase then addresses the organization of requirements over time, handling changes, maintaining traceability between requirements and downstream artifacts, and ensuring alignment with evolving stakeholder needs and project goals. *Validation and Verification (V&V)* follows, ensuring that requirements accurately reflect stakeholder intent (*Validation*) and that system specifications meet these requirements (*Verification*). In a waterfall-like, idealised software life-cycle, these activities are followed by other SE phases, which heavily depend on requirements, such as *Architecture and Design*, *Implementation*, and *Testing*.

RE phases typically consist of one or more NL-intensive tasks. These include *Classification*, i.e., categorising requirements into different classes such as functional vs quality, *Tracing*, i.e., defining trace-links between requirements and other artifacts, or *Defect Detection*, i.e., identifying quality issues in requirements. Traditionally, RE tasks revolve around the artifact known as the *System/Software Requirements Specification*, which provides a structured and comprehensive account of the system’s intended functionality, quality attributes, and constraints. The SRS serves as a contractual reference among stakeholders, guiding subsequent phases of software development, from design to testing and maintenance. Its role is twofold: on the one hand, it ensures that stakeholder needs are captured, agreed upon, and communicated in a precise manner; on the other hand, it provides a stable foundation against which system implementation and verification can be conducted. However, in recent years, a broader range of artifacts has entered the RE landscape, complementing the traditional reliance on specifications. These include *User Stories*, which capture functional needs in lightweight, scenario-driven forms; *User Feedback* collected from social media platforms and app stores, offering direct insights into user satisfaction and pain points; *Legal and Regulatory Documents*, which introduce compliance constraints; and other forms of unstructured or complex artifacts. Together, these sources expand the scope of RE beyond classical specification, but also introduce new challenges of interpretation, integration, and consistency management.

2.2. LLMs

LLMs represent a class of sophisticated artificial intelligence (AI) systems designed to handle and generate human language in a wide range of contexts. They are trained on extensive corpora of text—often encompassing diverse sources such as books, websites, academic papers, and social media—to learn patterns, structures, and relationships within language. By internalizing these patterns, LLMs can effectively capture semantic nuances, syntactic rules, and contextual cues that allow them to perform various NLP tasks. Among the most prominent examples of LLMs are models like GPT [15], which has demonstrated remarkable capabilities in tasks such as text completion, idea generation, and conversational interaction.

These models typically rely on deep learning architectures known as transformers [16], a key innovation that introduced the concept of self-attention. Self-attention mechanisms enable LLMs to assign different levels of importance to words (or tokens) in a given sequence, refining the way they capture contextual information. This ability is crucial for maintaining coherence across longer pieces of text, making models adept at tasks like summarization, translation, and question answering. Furthermore, LLMs can adapt to a variety of domains—ranging from legal and medical texts to creative writing—by leveraging transfer learning, where a model pre-trained on extensive general data is fine-tuned on more specialized or smaller datasets.

A notable characteristic of LLMs is their ability to generate coherent and contextually relevant text outputs, even in scenarios that require nuanced understanding. Their scalability allows for handling vast amounts of data, thereby improving their predictions and reducing errors related to grammar, style, and factual consistency (though ongoing research continues to address challenges such as model bias and misinformation). As a result, LLMs have become essential tools in the modern NLP landscape, powering a variety of real-world applications: from chatbots and virtual assistants to intelligent tutoring systems, content moderation, and advanced research support. Given their versatility and potential for automation, LLMs are poised to revolutionize how we interact with and harness the power of language in technology-driven environments.

2.3. Scope of the Study: Conceptual Scheme

Fig.1 represents the main concepts and relations thereof used in this SLR, and that form the basis for our data extraction schemes. The de-

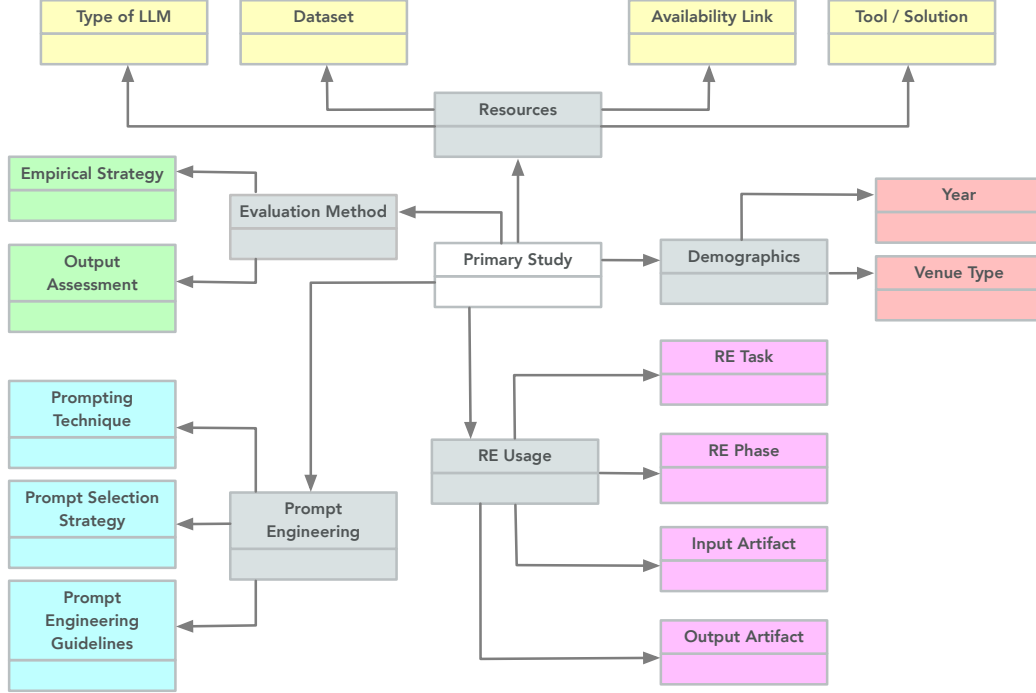


Figure 1: Conceptual scheme of LLM for RE used as the basis for data extraction.

tailed schemes are presented in each section of the data analysis part of the paper, to facilitate interpretation of the findings.

The central unit of analysis is the **Primary Study**, i.e., the empirical study that analyses the use of LLMs in RE. The primary study has **Demographics**, including in particular Year and Venue Type, besides other typical information (authors, DOI, etc.)—not represented in the figure.

A main element of interest is the **RE Usage**, i.e., how the study uses LLMs in RE. This entails addressing a specific **RE Task**, for example, requirements classification or defect detection, which can be applied during a certain **RE Phase**, e.g., elicitation, analysis, modelling. The task typically take a main **Input Artifact**, e.g., a software requirements specification, and produces an **Output Artifact**, e.g., a graphical model⁶. Note that the same

⁶More than one artifact can be taken as input, and more than one can be produced as output. Here, for simplicity, we consider the most relevant artifact considered in the primary studies

RE task can be applied in different phases and with different artifacts. For example, the classification task can be applied during the RE phase of elicitation, for example using app reviews as input, or during the management phase, using software requirements specification as input.

To trigger the generative behaviour of the LLM, the study applies a certain **Prompt Engineering** approach. This consists of **Prompting Techniques**, e.g., zero-shot, few-shot—typically more than one is used in a paper. Furthermore, it includes a strategy to select the most appropriate phrasing to be used for the prompt (**Prompt Selection Strategy**), and it can include a reference to the **Prompt Engineering Guidelines** adopted.

The study also uses a set of **Resources**. These are characterised by the **Type of LLM** adopted, often more than one, and the **Dataset** used for the empirical evaluation. Furthermore, the tool can make available through a link (**Availability Link**) the source code used and the prompts. Finally, it could incorporate the LLMs in a more complex **Solution**, e.g., combining both LLMs and standard NLP techniques to perform a certain RE Task, or an integrated **Tool** that implements the full solution⁷.

A primary study uses **Evaluation Methods**. This includes the **Empirical Strategy** [17] for the design of the study, for example, a laboratory experiment comparing different LLMs, or an experimental simulation, comparing users’ performance on a certain task with and without LLMs. The output generated by the LLM is typically assessed (**Output Assessment**), and this can be done, for example through quantitative methods, e.g., applying precision and recall, or through qualitative ones, for example through thematic analysis [18] of the generated output.

3. Related Reviews

The integration of LLMs into the field of SE has shown transformative potential across various tasks. A foundational survey by [19] highlights LLMs’ emergent properties, such as novelty and creativity, enabling their use in coding, design, repair, refactoring, performance improvement, documentation, and analytics. It emphasizes challenges like mitigating incorrect outputs and

⁷Often, in the surveyed papers, it is unclear whether a certain solution has been actually implemented in an integrated tool. For this reason, in this paper, we treat the two types of resources together.

advocates for hybrid approaches combining LLMs with traditional SE methods to ensure reliability. Still encompassing the whole SE field, Hou *et al.* [20] conducted an SLR of 395 publications from 2017 to 2024, categorizing LLM applications in SE, examining data collection, optimization, and evaluation strategies, and identifying supported tasks, while noting gaps for future research. Jin *et al.* [21] go beyond the study of LLMs in isolation and consider LLM-based agents, that is, systems where LLMs are enriched with tools, memory, and decision-making mechanisms to perform more complex and autonomous tasks. Additionally, He *et al.* [22] explores LLMs in multi-agent systems for SE, conducting a systematic review of their applications across the applications, with case studies illustrating the frameworks’ capabilities and limitations, and proposing a research agenda to enhance agent synergy and autonomy for Software Engineering 2.0.

Some studies have also focused on specific subfields of SE. In software testing, Wang *et al.* [23] analyzed 102 studies, showing LLMs’ roles in test case preparation and program repair, discussing prompt engineering, scalability challenges, and innovation opportunities. LLMs also have an impact on software architecture, automating and enhancing tasks such as design decision classification, pattern detection, and architecture generation. A systematic review [24] analyzed 18 articles and found that LLMs excel in these areas but are underutilized in tasks like code generation and architecture conformance checking, highlighting opportunities for future research to expand their application in software architecture.

Overall, these studies demonstrate the growing interest in applying LLMs to software engineering. However, none of them specifically target the RE field. An extensive literature review of 404 studies was conducted by Zhao *et al.* [25], covering research up to 2019 that applied NLP techniques to RE. Our work can be seen as a natural continuation of that effort, tracing the evolution from traditional NLP approaches to the emerging use of LLMs in RE.

4. Research Methodology

Our study followed the guidelines proposed by Kitchenham [26]. We first defined the research questions based on the overall research goal. Next, we drafted a review protocol to guide data collection from primary studies. We then performed a literature search and selected studies using inclusion and exclusion criteria. After that, we read the selected studies, extracted

relevant data items, and recorded them in data extraction forms. Finally, we consolidated the information and reported it in our study.

4.1. Research Questions

The overall goal of our SLR is *to provide a broad overview of the application of LLMs for RE* by categorising and summarising the existing literature. Based on this goal, we formulated the following research questions:

- *RQ1*: What are the demographics of publications on LLMs for RE?
- *RQ2*: How are LLMs used in RE?
 - *RQ2.1*: What RE tasks do LLMs support?
 - *RQ2.2*: What RE phases do LLMs address?
 - *RQ2.3*: What input artifacts are used?
 - *RQ2.4*: What output artifacts are produced?
 - *RQ2.5*: What is the co-occurrence of input and output artifacts?
- *RQ3*: How is prompt engineering applied in LLM-based RE approaches?
 - *RQ3.1*: What prompt engineering techniques are employed?
 - *RQ3.2*: What prompt selection strategies are used?
 - *RQ3.3*: What types of prompt engineering guidelines are followed?
- *RQ4*: What resources are used in LLM-based RE approaches?
 - *RQ4.1*: What datasets are used?
 - *RQ4.2*: Are source code and prompts publicly available?
 - *RQ4.3*: What types of LLMs are used
 - *RQ4.4*: Is a novel tool or solution incorporating LLMs proposed?
- *RQ5*: How are LLM-based RE approaches evaluated?
 - *RQ5.1*: How are the outputs of the LLMs assessed?
 - *RQ5.2*: What primary empirical evaluation strategies are used?

In answer to RQ1, we report the number of studies published over time and characterize their types and the venues where they have been published. We also highlight the most frequent venues where scholars have published their work, listing key contributions in this research area. With RQ2, we aim to understand the application of LLMs in RE: which tasks and phases they support, what types of input artifacts are provided to them, what output artifacts they generate, as well as their co-occurrence. RQ3 investigates how prompt engineering is applied in LLM-based RE applications, including techniques used, prompt selection strategies, and guideline types reported in the primary studies. In RQ4, we explore availability of supplementary materials of these studies to support of future replication and extensions studies. More specifically the questios explore the availabilty of used dataset, source code, prompts, LLM types as well as information about using the novel tooling for LLM-based RE approaches. Finally, RQ5 explores how LLM-based RE approaches are evaluated in the literature, focusing on the empirical strategies used and how LLM outputs are assessed.

4.2. Literature Search and Selection

We followed a systematic literature search and selection strategy to identify relevant studies focusing on the application of LLMs to RE. The process was conducted in two main stages: an initial digital library search and a complementary issue-to-issue search. Figure 2 summarises the main steps of this process and their outcomes. All steps were performed by the first author of the paper, who double-checked with the last author in case of doubts⁸.

Primary Search in Digital Libraries. The primary search was conducted using *Scopus*⁹ digital library, chosen for its comprehensive coverage of high-quality journals and conference proceedings from reputable publishers such as Elsevier, IEEE, and ACM. We formulated a search query that we applied to both the meta-data and full-text (when available) of the publication. The query was formulated using major terms (and their synonyms) referred to in our research goal. The adopted string is as follows.

⁸We would like to remark that this is a preliminary version, and in a further, more extensive publication we will perform cross-checks between multiple authors to increase the reliability of the findings.

⁹<https://www.scopus.com>

```

("large language model" OR LLM* OR "prompt engineering" OR
"prompt tuning" OR prompt*) AND ("requirements elicitation"
OR "requirements analysis" OR "requirements specification"
OR "requirements modeling" OR "requirements validation" OR
"requirements verification" OR "requirements management" OR
"requirements traceability" OR "requirements classification"
OR "requirements document" OR "requirements engineering")

```

We used LLM-representative terms in conjunction () with RE-related terms. In the first set, we also considered the terms associated with prompt engineering, as this is a primary element that distinguishes the use of LLMs from non-generative models, at least in the terminology typically adopted in RE. Furthermore, in preliminary searches, we identified several papers employing the term ‘prompt’ in its different variations. For RE-related terms, we adopted the same ones used by Zhao et al. [27] in their SMS on NLP for RE.

We executed the query on September 11, 2024, which returned 244 studies. We then screened abstract and titles for relevance. This resulted in 136 papers, which were then analysed according to the inclusion and exclusion criteria (see Table 2). After applying the criteria, 62 studies were kept for further analysis.

Secondary Venue-Based Search. To ensure broader coverage and minimise the risk of omitting relevant studies, we conducted a secondary search that specifically targeted key journals and conferences in RE. This approach helps address potential limitations in terminology, indexing, or database coverage. The search focused on primary studies published between 2020 to 2024 in major SE/RE conferences and journals. Table 1 illustrates the selected venues for manual search. On December 18, 2024, we performed the secondary search using publications from these venues. After removing duplicates, screening them, and after using the same inclusion and exclusion criteria as in the initial search, 12 additional papers were selected—the limited number confirms the quality of the search string adopted for primary search. Our literature search resulted in 74 studies that we read, analysed and categorised.

Table 1: Selected conference proceedings and journals for manual search

Requirements Engineering Journal	REJ
IEEE Transactions on Software Engineering	TSE
ACM Transactions on Software Engineering and Methodology	TOSEM
Information and Software Technology	IST
Journal of Systems and Software	JSS
Empirical Software Engineering	EMSE
International Requirements Engineering Conference	RE
Requirements Engineering: Foundation for Software Quality	REFSQ

Table 2: Inclusion and exclusion criteria for study selection

I1	Primary studies using LLMs to support RE tasks.
I2	Peer-reviewed studies published at conferences, workshops or journals
I3	Papers in conferences ¹ , Journals ² , and workshops in CORE A*/A/B conferences ³ .
E1	Papers not written in English.
E2	Papers using LLMs without the purpose of supporting RE.
E3	Secondary and tertiary studies, short papers (less than 5pp) and book chapters.
E4	Conceptual papers without empirical evaluations.

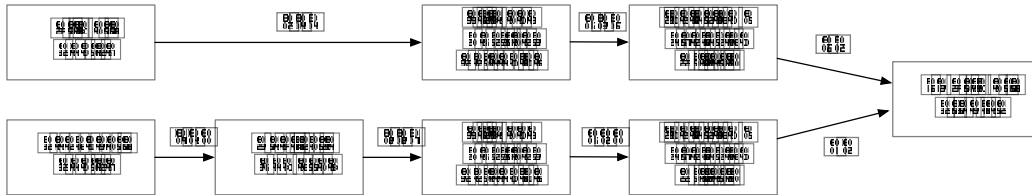


Figure 2: Systematic search process: papers retrieved from the primary and secondary search and filtered through inclusion and exclusion criteria.

¹<https://portal.core.edu.au/conf-ranks/>

²<https://www.scimagojr.com/journalrank.php>

³For B venues, we screened the workshop studies, and considered only those that had a quality comparable to those of workshops in A*/A venues.

4.3. Data Extraction and Synthesis

The first and last authors created a data extraction form to collect detailed information from each of the selected studies, and created classification schemes for each data field in the form. The fields in the form mirrored the RQs, and can be mapped one to one with the elements of the conceptual representation presented in Sect. 2.3.

To construct our classification schemes for each of the field, we adopted a structured yet adaptive approach to categorise various dimensions of our study. The scheme development process was informed by existing literature and enriched with our domain expertise to ensure both their alignment with the current state of research and their continued relevance. Specifically, if the literature included a classification scheme that was deemed appropriate for the specific data field, considering the retrieved literature, such classification scheme was adopted or minimally adapted considering possible variants emerging during the analysis of the papers. Otherwise, if no representative scheme could be retrieved from the literature, a novel scheme was defined by the authors inductively. During paper analysis, narrative information was initially reported in the data extraction form to characterise the data field of interest. Then, this narrative information was thematically analysed and aggregated into classes by the first and last author. These classes formed the final data extraction scheme for the specific field.

In Table 3, we list the extracted fields, together with the associated RQs, and the reference to their classification schemes. These latter are reported directly in the results section, to make it easier for the reader to interpret the results. The last column of the table refers to the source of the classification scheme, if any. It reports 'ind.' when the inductive approach was adopted, or '-', in case no classification scheme was required due to the nature of the field.

Table 3: Data extraction form. Sch. = Classification scheme or Table with results, in case the scheme was not required. Ref. = Reference for the scheme definition. 'ind' = defined by induction. '-' = not required or trivial.

Sch.		Ref.	
Authors, Title, Year, Citation key, Venue Type, DOI			
Authors, Title, Year, Citation key, Venue Type, DOI	capture the bibliographic information of each study, including its identity, publication details, and contributing researchers. In our mapping study we use only year and venue type, i.e., conference/journal/workshop for classification purposes.	-	-
RE Task			
RE Task	defines the RE task being addressed (e.g., elicitation, validation, traceability).	5	[28]
RE Phase			
RE Phase	refers to the RE lifecycle stage covered (e.g., elicitation, specification, analysis, management).	6	[27]
Input Artifact			
Input Artifact	identifies the data or artifacts used as input (e.g., requirements specifications, user stories, models) for the LLM.	7	[27]
Output Artifact			
Output Artifact	specifies the type of output generated (e.g., structured requirements, traceability links, classification labels).	8	[27]
Prompting Technique			
Prompting Technique	describes the general strategies used for prompting (e.g., zero-shot, few-shot, chain-of-thought).	10	ind.
Prompt Selection Strategy			
Prompt Selection Strategy	explains how initial prompts are chosen (e.g., predefined templates, trial and error).	11	ind.
Prompt Engineering Guidelines			
Prompt Engineering Guidelines	outlines any frameworks, best practices, or structured guidance applied in prompt engineering.	12	ind.
Types of LLM			
Types of LLM	lists the specific LLMs utilized in the study (e.g., GPT, Llama).	14	ind.
Availability Link			
Availability Link	link to the source code and/or prompts shared for reproducibility purposes.	A.18	-
Dataset			
Dataset	describes the dataset(s) used, including whether they are public or proprietary and how they are referenced.	13	-
Tool/Solution			
Tool/Solution	specifies the specific name or identifier of the solution/tool, if available.	15	-
Empirical Strategy			
Empirical Strategy	indicates what type of empirical strategy was adopted in the study, (e.g., laboratory experiment, experimental simulation, etc).	17	[17]
Output Assessment			
Output Assessment	describes how the outputs generated by the LLM are evaluated or interpreted, (e.g., qualitative methods, quantitative methods, or both).	16	ind.

Data extraction was performed by the first author. Similarly to the search phase, we did not systematically check the reliability of the data extraction, but the first author consulted with the last author in case of doubts. A more systematic cross-check will be performed in future versions of the paper when this research will be submitted to a journal.

5. Results

We now present the results of our mapping study, addressing research questions RQ1–RQ5. These questions were answered using information collected from primary studies and by adopting classification schemas (see Sect. 4).

5.1. RQ1: Demographics

A total of 74 primary studies on the use of LLMs for requirements engineering were published between 2023 and 2024. Although the search time had no boundary, no paper in the field referred to LLMs before 2023 according to our search results. Despite the fact that generative pre-trained transformers already existed already in 2018 with GPT-1, we argue that RE researchers started getting interested in LLMs when the ChatGPT phenomenon exploded in 2023—it was released on November 30, 2022. This suggests that the RE community, much like the general public, became enthusiastic about LLMs primarily when ChatGPT drew widespread attention to the technology. Unlike earlier models, which demanded significant technical expertise to access and operate, ChatGPT provided an easy-to-use interface that suddenly made advanced generative capabilities available to anyone, including RE researchers. Fig. 3 shows the annual distribution of these publications. The number of studies increased significantly in 2024 compared to 2023 by 30 studies (a 136% rise), indicating growing research interest in the LLM4RE area. Interestingly, the total number of papers published in the entire history of NLP for RE (NLP4RE), dating back to 1983 and surveyed by Zhao until 2019, is 404. In just two years, publications in LLM4RE already account for almost a fifth of that number. If this trend continues, by 2026, the number of LLM4RE papers could equal the total NLP4RE publications accumulated over roughly 40 years.

The primary studies are distributed across three types of publication venues: conferences, workshops, and journals. Fig. 4 presents the numerical distribution of these studies by venue type. Most publications appeared in conferences (49 studies, 66%), followed by workshops (17 studies, 23%) and journals (8 studies, 11%), indicating that LLM4RE is still in an exploratory stage, with findings more often presented in conferences and workshops than in archival journals.

Table 4 presents the top venues where more than two papers appeared. We see that the majority of the papers are at the flagship RE conference,

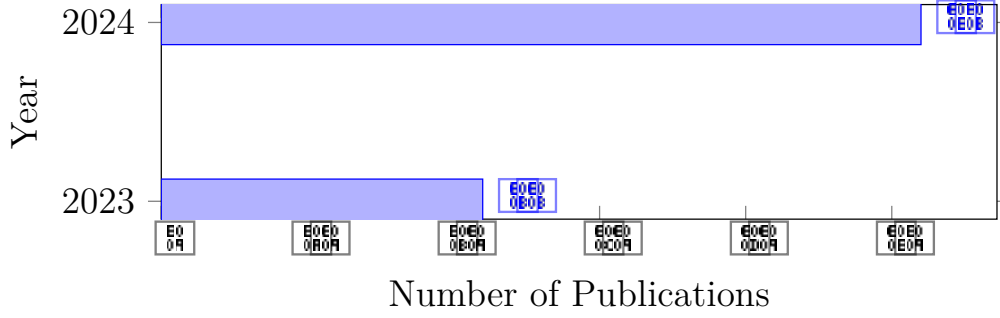


Figure 3: Number of publications per year

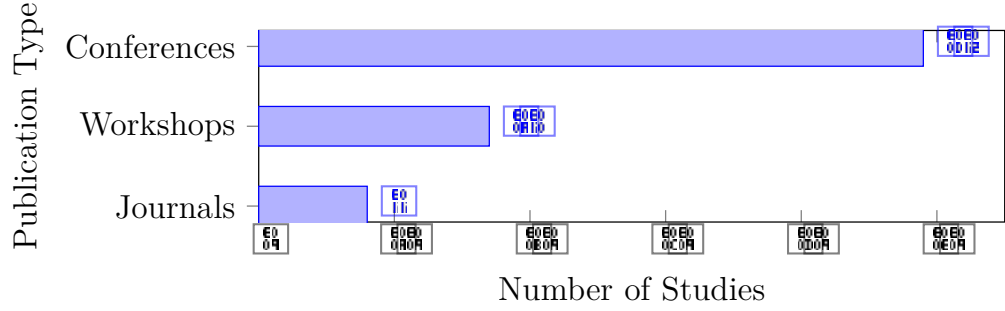


Figure 4: Distribution of research papers per venue type.

i.e., RE (21 studies, 28%), followed by the conference’s workshop (15 studies, 20%). However, a large part of the studies (32 studies, 43%) appear in one or two venues only—not shown in the table. These venues include also HCII (International Conference on Human–Computer Interaction), LREC (Language Resources and Evaluation Conference), COLING (International Conference on Computational Linguistics), MSR (Mining Software Repositories), and CAV (International Conference on Computer-Aided Verification), which are not typical RE venues, and some even not SE ones. This indicates that there is a growing interest in requirements-related tasks and artifacts, even outside traditional RE and SE circles. It represents an opportunity for RE researchers to reach out to a broader set of communities—such as AI, NLP, and HCI—both to showcase domain-specific challenges and to benefit from cross-fertilization with methods and perspectives developed outside the RE field.

Table 4: Most common venues (# = number of studies).

	
IEEE International Requirements Engineering Conference (RE)	21
Requirements Engineering Conference Workshops (REW)	15
IEEE Transactions on Software Engineering (TSE)	3
Model-Driven Requirements Engineering (MoDRE)	3
Other Venue	32

RQ1: Demographics

- The surge of 74 studies starting only in 2023 shows that the RE interest in LLMs began only after the ChatGPT release. RE researchers, like the general public, engaged with LLMs once easy-to-use interfaces lowered technical barriers.
- Publications rose 136% in 2024 vs 2023, signaling rapidly growing momentum. In just two years, LLM4RE papers reached 20% of all historical NLP4RE work (1983–2019). If growth continues, LLM4RE output may equal NLP4RE by 2026.
- Most studies appear in conferences (66%), fewer in workshops (23%) and journals (11%), indicating LLM4RE as an early-stage research area, with findings disseminated quickly through conferences rather than journals.
- The flagship RE conference (28%) dominates, but 57% of papers are scattered across diverse venues. Papers also appear in non-RE/SE venues (e.g., HCII, LREC-COLING, MSR, CAV), showing cross-disciplinary interest. This trend is an opportunity for RE researchers to engage with broader communities (AI, NLP, HCI) for cross-fertilization of ideas.

5.2. RQ2: RE Usage

In this section, we explore how LLMs have been used in RE, considering different tasks, phases, as well input and output artifacts processed.

5.2.1. RQ2.1: RE Tasks

Table 5 presents a list of tasks and their descriptions in our study, along with a mapping to the 74 studies that investigated them, including the number and percentage of studies for each task. The tasks are categorized into

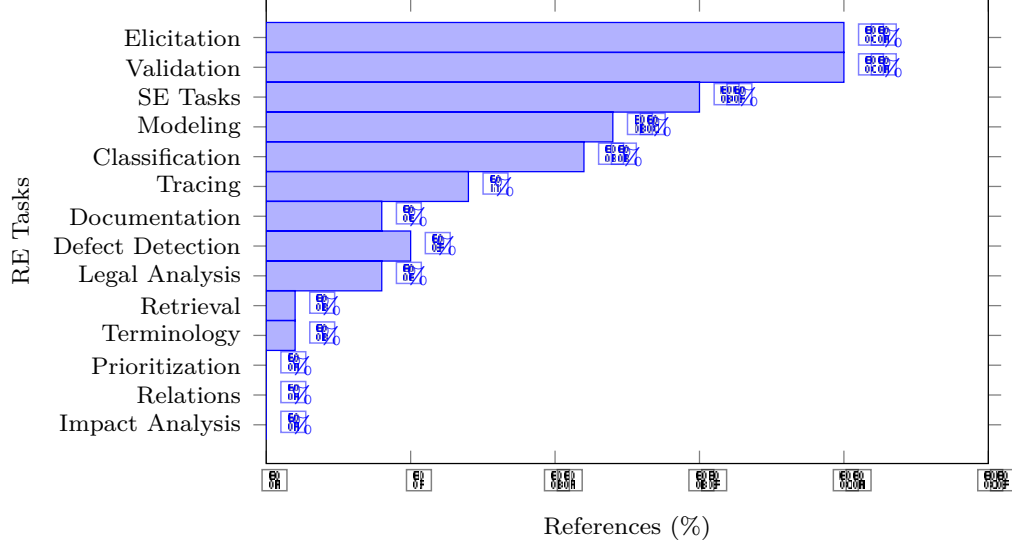
eleven categories, with each row listing the relevant references, the number of studies, and the percentage of the total they represent. The most frequently studied tasks are *Requirements Elicitation* and *Requirements Validation* (15 studies, 20% each), indicating strong research attention toward both the initial gathering and the later evaluation of requirements. Other well-represented areas include *Software Engineering Tasks* (11 studies, 15%), *Requirements Modelling* (9 studies, 12%), and *Requirements Classification* (8 studies, 11%), highlighting the importance of both the structural and analytical aspects of RE. Conversely, tasks such as *Requirements Retrieval* and *Terminology Extraction* are each addressed by only one study (1%), pointing to underexplored areas with opportunities for further investigation.

While a direct comparison with NLP4RE studies is not straightforward—since Zhao et al. [27] employed a different classification scheme—some interesting patterns emerge. In their survey, the most common tasks were Detection (i.e., *Requirements Defect Detection* in our categorization) and Classification. In contrast, in the LLM4RE landscape, these tasks rank only 8th and 5th, respectively. Instead, *Elicitation*, *Validation*, and *Modelling*—arguably the most human-intensive and inherently challenging tasks in RE—have become the focal point, reflecting the unique capabilities of LLMs to support more cognitively demanding activities. This shift suggests that LLMs are not merely automating traditional NLP4RE tasks but are enabling research to tackle tasks that were previously considered difficult for machines, potentially reshaping the priorities and opportunities in RE research. In this context, it is also noteworthy that a substantial number of studies focus on broader *Software Engineering Tasks*—for example, using requirements to generate test cases. This trend indicates that LLMs provide the RE field with the opportunity to move beyond its traditional boundaries and explore intersections with other software engineering sub-disciplines.

Table 5: RE-Oriented NLP tasks: descriptions, references, and distribution

Task	Description	References	Distribution
Requirements Elicitation	Gathering requirements from stakeholders through interviews, surveys, etc., to understand their needs, goals, and constraints.	[29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43]	15 20%
Requirements Documentation	Recording requirements in structured formats such as SRS or user stories.	[44, 45, 46, 47]	4 5%
Requirements Defect Detection	Identifying inconsistencies or quality issues in requirements early in the lifecycle.	[48, 49, 50, 51]	4 4%
Requirements Prioritization	Ranking requirements by importance and feasibility to guide implementation order.	—	—
Requirements Tracing	Linking requirements across artifacts for traceability and impact analysis.	[52, 53, 54, 55, 56]	5 7%
Requirements Classification	Organizing requirements (e.g., functional vs. non-functional) to aid analysis.	[48, 53, 57, 58, 59, 60, 61, 62]	8 11%
Requirements Retrieval	Reusing relevant requirements from prior projects or documents.	[63]	1 1%
Requirements Terminology Extraction	Extracting domain-specific terms to create glossaries.	[64]	1 1%
Requirements Relations Extraction	Identifying dependencies and associations between requirements.	—	—
Requirements Modeling	Representing requirements in abstract forms (e.g., UML diagrams).	[65, 66, 67, 68, 69, 70, 71, 72, 73]	9 12%
Requirements Validation	Ensuring that requirements reflect stakeholder goals and expectations.	[74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88]	15 20%
Change Impact Analysis	Assessing the effect of proposed changes on existing requirements.	—	—
Software Engineering Tasks	Applying requirements to support downstream SE tasks (e.g., testing, coding).	[89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99]	11 15%
Legal Requirements Analysis	Integrating legal/compliance constraints into RE through systematic analysis.	[52, 100, 101, 87]	4 4%

Figure 5: RE tasks and references



5.2.2. RQ2.2: RE Phases

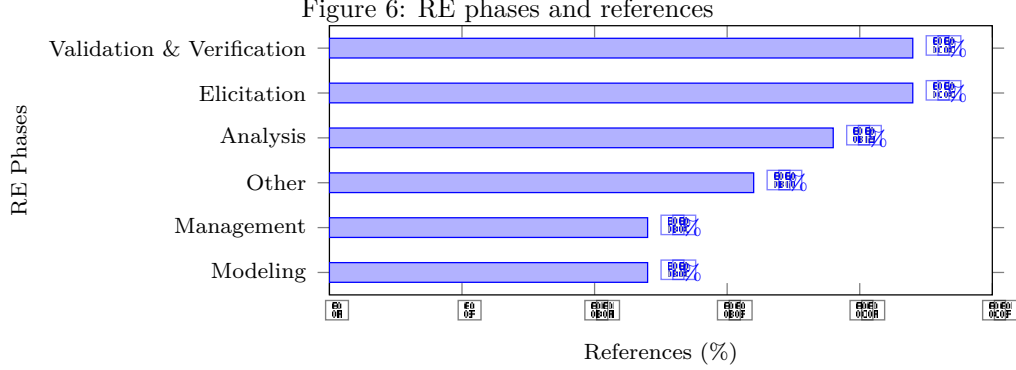
Table 6 presents six major RE phases, their definition and the distribution of primary studies across these phases, along with references to the studies. The studies are grouped into six high-level RE phases based on their primary focus. The most represented phases are *Elicitation* and *Validation & Verification (V&V)* (16 studies, 22% each), indicating a strong emphasis in the literature on both the initial capture and the subsequent evaluation of requirements. The *Analysis* phase is also well represented (14 studies, 19%), reflecting active research on interpreting, refining, and classifying requirements. Both *Modeling* and *Management* are addressed by 9 studies each (12%), suggesting steady but comparatively lower attention to the structuring and ongoing handling of requirements. The *Other* category (12 studies, 16%) includes studies focused on broader or cross-cutting concerns that do not align specifically with a single RE phase. Overall, this landscape suggests that LLMs are not limited to specific stages but are being leveraged throughout the RE process, supporting both phase-specific and cross-cutting activities.

Again, comparing our results with those of Zhao et al. [27] offers additional insights. In their survey, *Analysis* was by far the most commonly addressed phase (42.7%), while *V&V* accounted for only 4% of the studies, and *Elicitation* represented 16.76%. In contrast, the landscape in LLM4RE

is completely different: as observed for the tasks, the first and last phases of the RE lifecycle have gained prominence. On the one hand, this shift opens opportunities for automating highly relevant and time-consuming RE activities. On the other hand, caution is warranted, as these phases are particularly delicate—they involve direct interaction with stakeholders, and $V\&V$, in particular, can be critical when compliance with standards or regulatory requirements is at stake. Thus, while LLMs offer significant potential, careful consideration of their role and limitations in sensitive phases of the RE process remains essential.

Table 6: RE phases used in this research (adapted from [25]): definitions, references, and frequencies

RE Phase	Definition	References	Frequency	Percentage
Elicitation	Activities that help understand goals, objectives, and stakeholder needs for building a proposed software system.	[29, 30, 31, 40, 63, 32, 33, 34, 39, 35, 36, 38, 37, 41, 43, 42]	16	22%
Analysis	Evaluating the quality of recorded requirements and identifying anomalies like ambiguity, inconsistency, or incompleteness.	[52, 48, 100, 64, 49, 59, 101, 57, 58, 62, 61, 50, 51, 60]	14	19%
Modeling	Building conceptual models of requirements that are interpretable and structured.	[65, 66, 67, 68, 69, 70, 72, 71, 73]	9	12%
Management	Managing requirements across time and versions, and ensuring traceability among requirements and downstream artifacts.	[52, 44, 53, 54, 55, 46, 45, 47, 56]	9	12%
Validation & Verification	Ensuring requirements reflect stakeholder intent (validation) and proving that specifications meet them (verification), e.g., via model checking.	[74, 75, 78, 76, 77, 79, 80, 81, 82, 88, 101, 87, 83, 84, 85, 86]	16	22%
Other	Covers RE-related activities like test-case generation (“Testing”) or deriving design artifacts (“Design”).	[89, 90, 91, 102, 92, 93, 99, 94, 95, 96, 98, 97]	12	16%



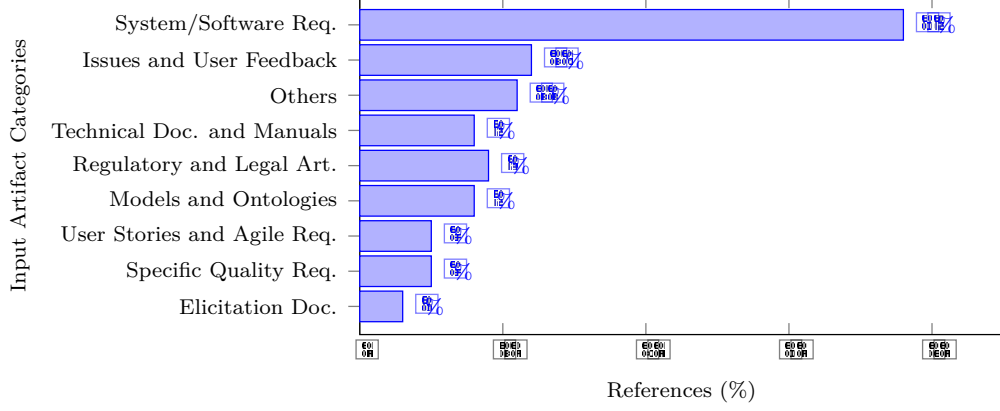
5.2.3. RQ2.3: Input Artifacts

Table 7 presents the nine categories of input artifacts identified in the surveyed literature, along with the distribution of input artifacts used in the analyzed studies and references to the corresponding papers. The studies were categorized into nine artifact types based on the primary source of input they addressed. The most common artifact type is *System/Software Requirements Specifications* (28 studies, 38%), indicating that a majority of research in LLM4RE focuses on traditional, structured requirement documents. *Issues and User Feedback* also account for a significant portion (9 studies, 12%), highlighting interest in mining user-reported content from platforms such as issue trackers and product reviews. Other moderately represented types include *Regulatory and Legal Artifacts* (7 studies, 9%), *Technical Documents and Manuals*, and *Models and Ontologies* (6 studies, 8% each), as well as *Others* (8 studies, 11%), the latter encompassing inputs that do not fit neatly into a single category. Less frequently addressed artifact types include *Specific Quality Requirements* and *User Stories and Agile Requirements* (4 studies, 5% each), and *Elicitation Documents* (2 studies, 3%). This distribution highlights the dominance of formal requirements specifications in current research, while also pointing to a growing interest in utilizing informal and diverse artifact sources for RE tasks. Also in Zhao et al. [27], the majority of the studies (60%) used *System/Software Requirements Specifications* as input artifacts. Here, the percentage has substantially decreased and there is a slightly more balanced distribution, suggesting that LLMs enable tackling not only novel tasks and phases but also interpreting novel types of documents.

Table 7: Input artifact categories: definitions, references, and frequencies

Category	Definition	References	Frequency
Regulatory and Legal Artifacts	Documents defining compliance, policies, contracts, and legal obligations for systems or projects.	[52, 100, 54, 82, 70, 94, 38]	7 9%
System/Software Requirements Specifications	Formalized documents listing the functional and non-functional requirements of a system.	[48, 44, 89, 74, 90, 75, 66, 91, 31, 76, 64, 40, 55, 99, 63, 80, 33, 34, 69, 59, 88, 101, 35, 71, 47, 51, 85, 83]	28 38%
Specific Quality Requirements	Requirements focusing on a specific system quality dimension, e.g., security or safety.	[53, 92, 79, 42]	4 5%
Elicitation Documents	Elicitation artifacts such as interview transcripts, focus groups, and workshops.	[29, 37]	2 3%
User Stories and Agile Requirements	Lightweight, iterative requirement descriptions commonly used in Agile methodologies.	[93, 68, 32, 73]	4 5%
Technical Documents and Manuals	Detailed technical documentation for implementation, usage, and maintenance.	[78, 102, 49, 98, 97, 86]	6 8%
Issues and User Feedback	App reviews, crowd requirements, and issues from platforms such as GitHub, Jira, etc.	[30, 81, 72, 87, 58, 45, 95, 61, 60]	9 12%
Models and Ontologies	Conceptual models and ontologies aiding requirements engineering and analysis.	[65, 77, 67, 39, 96, 43]	6 8%
Others	Other relevant artifacts or processes not categorized above.	[46, 57, 36, 62, 41, 84, 50, 56]	8 11%

Figure 7: Input artifact categories and references



5.2.4. RQ2.4: Output Artifacts

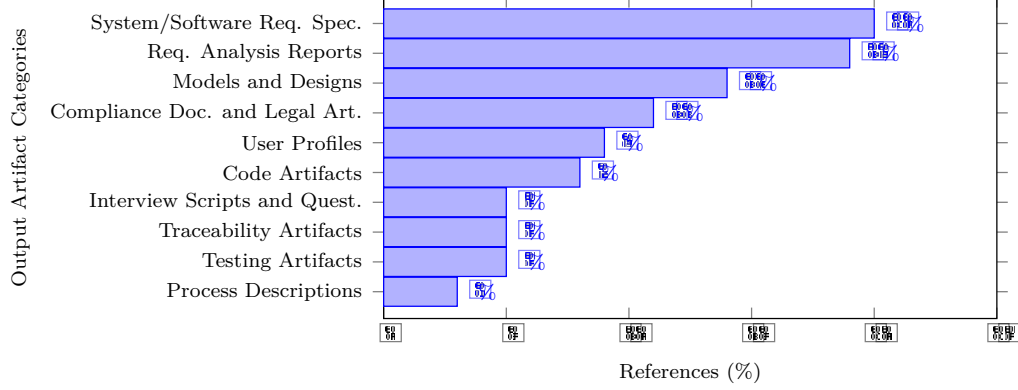
Table 8 presents the categorization of output artifacts identified in the primary studies, divided into ten types based on their nature. It also shows the distribution of these artifacts across the surveyed studies, along with references to the corresponding papers. The most frequently targeted artifact type is *System/Software Requirements Specifications* (15 studies, 20%), similar to the case of input artifacts. *Requirements Analysis Reports* are also highly represented (14 studies, 19%), indicating a strong focus on analytic—i.e., not purely generative—activities such as classification, refinement, and prioritization of requirements. *Models and Designs* appear in 10 studies (14%), in line with the increasing interest in model generation tasks. *Compliance Documents and Legal Artifacts* (8 studies, 11%) and *User Profiles* (7 studies, 9%) highlight efforts to address domain-specific and user-centric concerns. Several artifact types are addressed in a moderate number of studies, including *Code Artifacts* (6 studies, 8%), *Testing Artifacts* and *Traceability Artifacts* (4 studies, 5% each), and *Interview Scripts and Questionnaires* (4 studies, 5%). *Process Descriptions* are the least represented (2 studies, 3%). Overall, the table reveals that while much of the research focuses on core RE documents and analysis outputs, there is a growing effort to apply LLMs to a broader variety of RE outputs. In the study of Zhao et al. [27], output artifacts were not present. This marks the main difference between NLP4RE studies and LLM4RE studies: with LLM4RE, we can tackle novel, generative tasks that were not even possible before. At the same time, we can still tackle analytic tasks (i.e., those associated with *Requirements Analysis Reports*), showing

the high versatility of LLM-based solutions.

Table 8: Output artifact categories: definitions, references, and frequencies

Artifact Category	Definition	References	Frequency	Percentage
System/Software Requirements Specifications	Formalized documents listing the functional and non-functional requirements of a system.	[44, 90, 75, 64, 93, 49, 59, 57, 58, 45, 95, 71, 83, 50, 85]	15	20%
Process Descriptions	Textual descriptions of processes, methods, or procedures involved in system development.	[65, 87]	2	3%
Models and Designs	Visual or structural representations (e.g., diagrams, models) that support understanding or system design.	[66, 78, 68, 88, 101, 94, 47, 73, 41, 43]	10	14%
Code Artifacts	Source code and deliverables implementing system functions or refined from earlier iterations.	[91, 40, 63, 39, 35, 98]	6	8%
Requirements Analysis Reports	Reports analyzing consistency, contradictions, coverage, or other aspects of requirements.	[48, 74, 31, 53, 92, 55, 80, 33, 72, 46, 37, 84, 51, 86]	14	19%
Testing Artifacts	Test cases, mutation testing, prototypes, or validation-related documents.	[89, 32, 96, 97]	4	5%
Traceability Artifacts	Documents linking requirements to code or other outputs to support impact and change analysis.	[76, 77, 99, 56]	4	5%
Compliance Documents and Legal Artifacts	Legal/compliance documents such as GDPR checks, safety cases, contracts, or regulations.	[52, 100, 54, 102, 79, 34, 70, 38]	8	11%
Interview Scripts and Questionnaires	Structured elicitation tools like interview guides, question flows, and surveys.	[29, 67, 69, 82]	4	5%
User Profiles	Representations of user needs or values (e.g., personas, empathy maps, value stories).	[30, 81, 36, 62, 61, 60, 42]	7	9%

Figure 8: Output artifact categories and references



5.2.5. RQ2.5: Input and Output Artifacts Mapping

We analyze the co-occurrence between input and output artifacts across the reviewed studies to identify common pairs. Table 9 summarizes these pairs, while Fig. 9 shows a bubble plot of the co-occurrences. The most evident aspect is the high number of pairs (27 across 74 studies), which points to a fragmented research landscape pursuing widely different goals. This reflects the limited maturity of the field and highlights its exploratory nature, where academics are primarily investigating novel application opportunities in RE rather than converging on a set of well-defined objectives. On the other hand, it also highlights the versatility of the field, which can be seen as a playground for experimenting with diverse applications.

The most frequent pair is *System/Software Requirements Specifications-Requirements Analysis Reports* (6 studies, 8%). This confirms that the interest in purely analytic tasks is still present. The pair is followed by *Regulatory and Legal Artifacts-Compliance Documents and Legal Artifacts*. Regulatory compliance in RE is a lively research area, especially since software artifacts are being increasingly regulated, e.g., by GDPR¹⁰, or used as part of mission-critical products, such as healthcare devices, and transport systems that need to abide by high safety standards. We argue that, with the novel regulations such as the AI Act¹¹, these types of studies will become more and more popular. Interestingly, another common co-occurrence is

¹⁰<https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=celex:32016R0679>

¹¹<https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai>

System/Software Requirements Specifications-System/Software Requirements Specifications (5 studies, 7%). These studies typically focus on enhancing specifications, for example, by improving requirements completeness through question-answering or through automatic completion, or by translating requirements into well-defined templates. Despite the strong push toward exploring new applications, the fundamental concern for requirements quality continues to be at the core of RE research.

Input Artifacts	Output Artifacts	Studies	Count	Percentage
Regulatory and Legal Artifacts	Compliance Documents and Legal Artifacts	[52, 100, 54, 70, 38]	5	7%
System/Software Requirements Specifications	System/Software Requirements Specifications	[44, 90, 75, 64, 59]	5	7%
	Models and Designs	[66, 88, 101, 47]	4	5%
	Code Artifacts	[91, 40, 63, 35]	4	5%
	Requirements Analysis Reports	[48, 74, 31, 55, 80, 33]	6	8%
	Traceability Artifacts	[76, 99]	2	3%
Specific Quality Requirements	Requirements Analysis Reports	[53, 92]	2	3%
Elicitation Documents	Requirements Analysis Reports	[37]	1	1%
	Interview Scripts and Questionnaires	[29]	1	1%
User Stories and Agile Requirements	System/Software Requirements Specifications	[93]	1	1%
	Models and Designs	[68, 73]	2	3%
	Testing Artifacts	[32]	1	1%
Technical Documents and Manuals	System/Software Requirements Specifications	[49]	1	1%
	Models and Designs	[78]	1	1%
	Code Artifacts	[98]	1	1%
	Requirements Analysis Reports	[86]	1	1%
	Testing Artifacts	[97]	1	1%
Issues and User Feedback	System/Software Requirements Specifications	[58, 45, 95]	3	4%
	Requirements Analysis Reports	[72]	1	1%
	User Profiles	[30, 81, 61, 60]	4	5%
Models and Ontologies	Process Descriptions	[65]	1	1%
	Models and Designs	[43]	1	1%
	Code Artifacts	[39]	1	1%
	Traceability Artifacts	[77]	1	1%
Others	System/Software Requirements Specifications	[57, 50]	2	3%
	Requirements Analysis Reports	[46, 84]	2	3%
	Traceability Artifacts	[56]	1	1%

Table 9: Common input-output artifact mappings across primary studies.

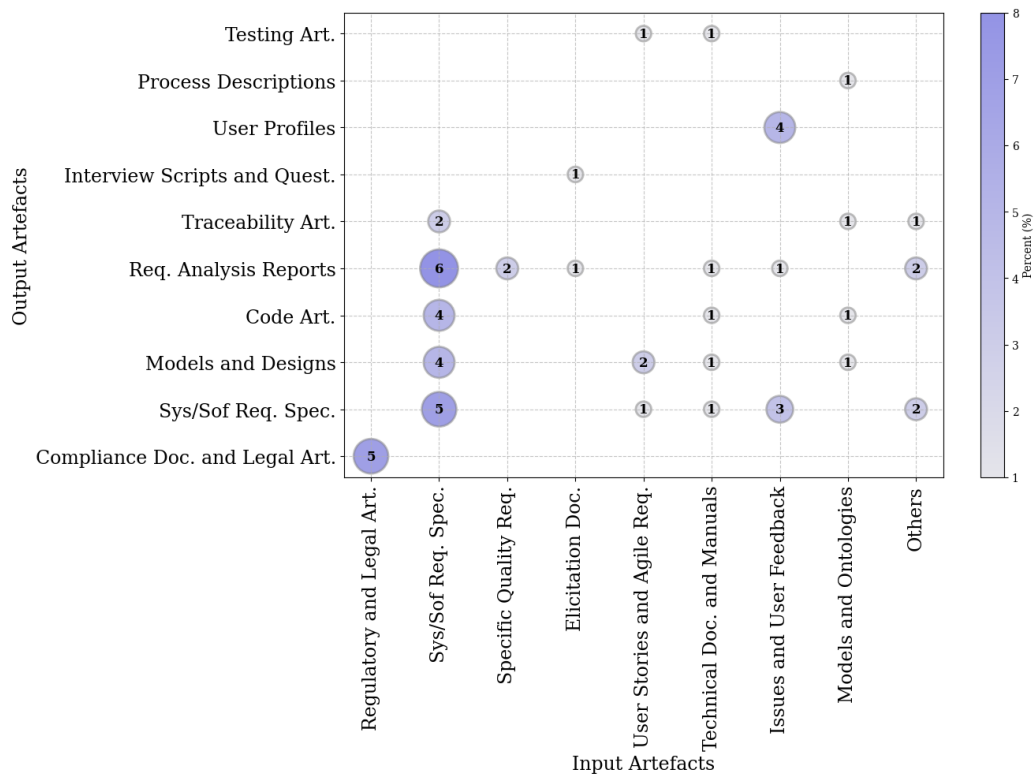


Figure 9: Bubble plot of input-output artifact mappings across primary studies.

RQ2: RE Support

- Unlike NLP4RE, where *Defect Detection* and *Classification* were central, LLM4RE research concentrates on cognitively demanding tasks such as *Elicitation* and *Validation* (20% each), as well as *Modeling* (12%) and broader *Software Engineering Tasks* (15%). This highlights the unique capacity of LLMs to support complex, human-intensive activities, and opens to greater integration with the broader software engineering field.
- Compared to earlier NLP4RE surveys where *Requirements Analysis* dominated, in LLM4RE studies *Elicitation and Validation & Verification (V&V)* are much more prominent. This shift opens automation opportunities but raises risks, since these phases involve stakeholder interaction and compliance-critical tasks.
- Most studies still use *System/Software Requirements Specifications* as input (38%), but there is a noticeable diversification: *Issues & User Feedback* (12%), *Regulatory & Legal Artifacts* (9%), as well as other less formal sources are gaining ground. This suggests LLMs make it possible to handle more complex and less structured artifact types.
- Beyond *Specifications* (20%) and *Analysis Reports* (19%), LLMs generate *Models*, *User Profiles*, *Compliance Documents*, and even *Code*, enabling tasks not present in earlier NLP4RE work, and showing the potential to radically expand the scope of the discipline.
- The field exhibits a fragmented landscape with 27 different input-output artifact pairs across 74 studies, underscoring its exploratory nature, but again confirming the versatility of LLMs.

5.3. RQ3: Prompt Engineering Approaches

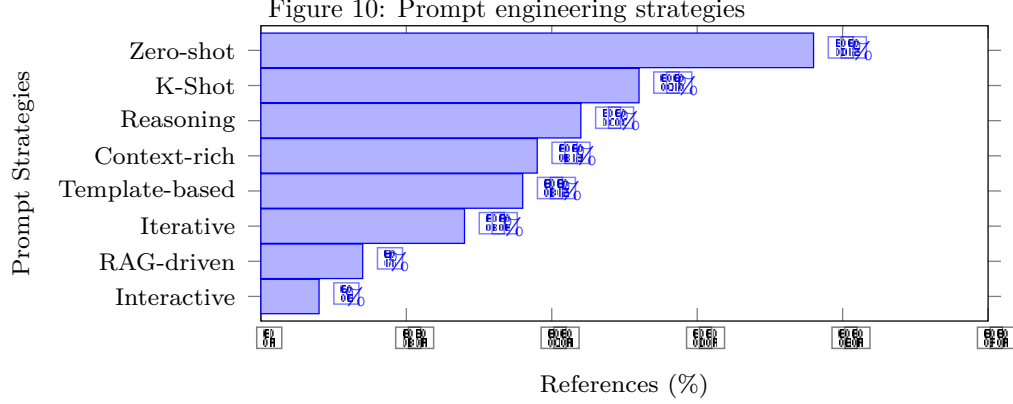
In this section, we examine prompt engineering strategies (e.g., *Few-shot* prompting), prompt selection strategies adopted to identify the prompts to be used in the study (e.g., *Trial and Error*), and prompt engineering guidelines. The results are reported along these three dimensions in the subsequent sections.

5.3.1. RQ3.1: Prompt Engineering Strategies

Table 10 presents the eight broad categories of prompt engineering strategies, together with definitions and distribution. Please consider that one paper can include more than one strategy. The most frequently used approach is *Zero-shot*, adopted in 28 studies (38%), followed by *K-shot* (*Few-shot* or *Many-shot*), reported in 19 studies (26%). The dominance of these simple strategies highlights a tendency to use LLMs “out of the box”, without specific prompt engineering or tuning efforts. The focus of these studies does not appear to be on prompting, but rather on answering the question “Can LLMs, even when used with minimal configuration, provide value in RE tasks?” This suggests that the field is still in an exploratory phase, where demonstrating feasibility takes precedence over optimizing techniques such as Prompt Engineering, Fine-tuning, or Domain Adaptation. However, also more complex strategies are also used, often together, or in comparison, with the simpler strategies. *Reasoning and Task Decomposition* techniques are employed in 16 studies (22%), *Context-rich*, which enriches inputs with additional background or metadata, appears in 14 studies (19%), followed by *Template-based* in 13 studies (18%). *Iterative*, involving multiple rounds of refinement or interaction, is used in 10 studies (14%). In contrast, the least frequently applied strategies are *RAG-driven* (5 studies, 7%) and *Interactive* (3 studies, 4%), suggesting that more dynamic or retrieval-augmented prompting remains underexplored in the current body of research. Overall, these results highlight a strong potential for exploring novel prompt engineering approaches in RE that move beyond the consolidated strategies currently in use. An extensive overview of prompting techniques, to be used as a map for exploration, is presented in [103].

Table 10: Prompt engineering strategies: definitions, references, and frequencies

Strategy	Definition	References	Frequency
Zero-shot	Prompting the model to perform a task without any examples or added context; relies solely on pre-trained knowledge.	[52, 48, 100, 90, 75, 30, 102, 53, 76, 64, 77, 92, 67, 79, 55, 80, 32, 81, 33, 72, 58, 45, 47, 38, 61, 37, 50, 89]	28
K-Shot: Few-shot, Many-shot	Including a few or many examples in the prompt to guide model behavior and output style.	[52, 89, 75, 53, 64, 93, 79, 55, 32, 81, 34, 69, 49, 59, 45, 51, 85, 57, 29]	19
Reasoning and Task Decomposition	Step-by-step reasoning or breaking tasks into subtasks within the prompt (e.g., Chain-of-Thought).	[52, 75, 66, 53, 92, 40, 67, 32, 33, 47, 88, 86, 56, 82, 96, 71]	16
Context-rich	Enriching prompts with background, metadata, or personas to better guide the model's responses.	[48, 44, 100, 90, 31, 54, 49, 84, 50, 60, 36, 42, 87, 46]	14
Template-based	Using predefined templates to structure the model's output format or style consistently.	[65, 74, 91, 31, 68, 101, 87, 98, 97, 43, 83, 95, 62]	13
Iterative	Prompting that involves multiple automatic refinement passes without human intervention.	[29, 64, 99, 63, 94, 39, 35, 73, 78, 41]	10
RAG-driven	Incorporating external knowledge into prompts using Retrieval-Augmented Generation (RAG) techniques.	[80, 69, 70, 82, 89]	5
Interactive	Prompts designed for back-and-forth interaction with the user (e.g., asking clarification questions).	[78, 96, 41]	3



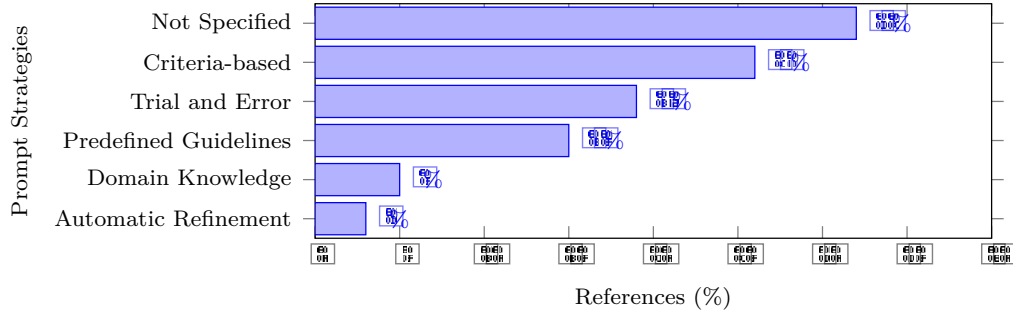
5.3.2. RQ3.2: Prompt Selection Strategies

Table 11 presents our results concerning preliminary prompt selection strategies. The most common category is *Not-specified*, observed in 29 studies (39%), indicating that a substantial portion of the literature does not provide explicit information about how prompts were selected or refined, showing limited sensitivity to the topic, despite the high impact of prompt formulation on the output of LLMs [104]. Among the explicitly described strategies, *Trial and Error*, involving manual and systematic refinement, was used in 15 studies (20%). *Criteria-based Refinement* appears in 19 studies (26%), showing that many studies use evaluation metrics or predefined goals to guide prompt design. Other strategies are less frequently reported. *Predefined Guidelines*, which rely on domain-specific or organizational standards, were used in 6 studies (8%). *Domain Knowledge-based* approaches, incorporating subject-matter expertise, appear in 4 studies (5%). *Automatic Refinement*, which involves programmatic or AI-driven improvement of prompts, was found in only 3 studies (4%), making it the least common approach. Overall, the data suggest that while structured strategies for prompt selection are emerging, a significant proportion of current research still relies on informal or undocumented practices. This suggests that there remains considerable room for improving LLM4RE performance, as the full potential of prompt engineering has yet to be unlocked.

Table 11: Prompt selection strategies: definitions, references, and frequencies

Trial and Error (Manual Systematic Refinement)	Iterative experimentation and manual adjustment of prompts based on observations, feedback, and performance.	14	19%
	[89, 30, 93, 67, 80, 33, 34, 49, 82, 72, 39, 87, 96, 54]		
Automatic Refinement (Systematic Refinement)	Automated methods (e.g., algorithms, feedback loops) for optimizing prompts without human intervention.	2	3%
	[99, 70]		
Criteria-based Refinement	Refines prompts using specific criteria such as evaluation metrics or goal alignment.	19	26%
	[52, 48, 74, 100, 66, 31, 76, 79, 32, 69, 59, 101, 58, 95, 71, 47, 38, 61, 41]		
Predefined Guidelines	Refines prompts based on structured standards, templates, or documented best practices.	11	15%
	[105, 29, 106, 64, 77, 92, 73, 60, 68, 56, 107]		
Domain Knowledge Based	Refines prompts using domain-specific insights or subject-matter expertise.	4	5%
	[90, 91, 55, 45]		
Not Specified	No explicit information provided on how prompts were selected or refined.	24	32%
	[44, 75, 78, 102, 53, 40, 63, 81, 88, 94, 46, 35, 36, 62, 37, 98, 97, 43, 83, 84, 50, 51, 85, 86]		

Figure 11: Prompt selection strategies and references



5.3.3. RQ3.3: Prompt Engineering Guidelines

Table 12 lists the guidelines along with their associated references, number of studies, and corresponding percentages out of the 74 total studies. The most commonly cited source is *OpenAI’s prompt engineering guidelines* (6 studies), followed by *White et al.* [108] (3 studies). *Mohedas et al.* [105], *Arvidsson & Axell* [106], *Blaha & Rumbaugh* [109], and *Ekin et al.* [107] were each cited in 1 study. This indicates that only a small subset of studies (11 studies, 15%) explicitly referenced the use of formal prompting guidelines. The results suggest a gap in standardized practices, with the majority of studies relying on informal or undocumented prompt design processes.

Table 12: Prompting guidelines cited in primary studies (11 of 74 studies, 15%)

Guideline	Reference	Number of Studies	Percentage
Mohedas et al. (2022) [105]	[29]	1	1%
Arvidsson & Axell (2023), Arora et al. (2023) [106]	[64]	1	1%
OpenAI’s prompt engineering guidelines [105]	[77, 92, 73, 60, 64, 105, 106, 107]	8	11%
Blaha & Rumbaugh (2005) [109]	[68]	1	1%
White et al. (2023) [108]	[66, 68, 56]	3	4%
Ekin et al. (2023) [107]	[56]	1	1%

RQ3: Prompt Engineering Approaches

- In explored studies, prompt engineering predominantly employs simple strategies, with *Zero-shot* used in 38% of studies and *K-shot* in 26%, reflecting a focus on evaluating LLMs’ feasibility for RE tasks with minimal configuration. More advanced techniques, such as *Reasoning and Task Decomposition* (22%), *Context-rich* (19%), and *Template-based* (18%), are less frequent, while *RAG-driven* (7%) and *Interactive* (4%) remain underexplored.
- Prompt selection is often undocumented (39% *Not-specified*), with *Trial and Error* (20%) and *Criteria-based Refinement* (26%) being the most common strategies. This indicates that there is still significant potential for enhancing LLM performance through more sophisticated and structured prompt engineering approaches.

5.4. RQ4: Resources

We now report information on the resources used in the primary studies, including datasets, available source code and prompts, and the types of LLMs adopted.

5.4.1. RQ4.1 Datasets

Table 13 provides an overview of the datasets identified for our mapping study. In total, 10 publicly available datasets were extracted from the selected studies. These datasets were referenced by 12 out of the 74 primary studies reviewed (16%). Each entry includes the dataset name, a brief description of its content and scope, the specific RE task(s) it supports, and the corresponding primary studies that make use of the dataset. The table highlights the diversity of datasets across various RE tasks, such as *Regulatory Compliance* [52], *Defect Detection* [48], *Requirements Modeling* [66, 67, 68], *Requirements Validation* [74, 77], *Requirements Elicitation* [40], and *Software Engineering Tasks* [93]. For datasets associated with multiple RE tasks or studies e.g., the *PURE* dataset [102, 66, 90], rows are subdivided to associate each RE task with its corresponding primary study. All datasets listed are publicly available and include persistent links to support accessibility and reproducibility. It should be remarked that only 16% of the studies made use of public datasets, while many of the studies relied on private ones. This can create a replicability problem and further exacerbate the issues raised for the NLP4RE field [110], which include difficulty in reproducing results and comparing performance across different baselines, which are the basis for any scientific discipline.

Dataset Name	Description	Primary Task	Reference
Annotated dataset for regulatory change ^[98]	Data containing the annotated dataset for regulatory change analysis	Legal Requirements Analysis	[52]
GDPR compliance decisions dataset ^[99]	Dataset supporting classification of software requirements, specification defect identification, and conflict detection across various sources	Defect Detection	[48]
UAV Dataset	Features 400 entities across classes, properties, and individuals, with 66,296 sentences from Wikipedia and development forums	Automatic text completion for software requirements	[44]
PURE dataset ^[100]	Contains 79 public natural language requirements documents from the web	Standards Compliance	[102]
		Requirements Modeling	[66]
		Requirements Analysis	[90]
PROMISE NFR ^[101]	Contains 625 sentences: 255 functional and 370 non-functional requirements	Requirements Validation	[74]
Rico dataset ^[81]	Mobile app UI dataset for building data-driven design applications	Requirements Validation	[77]
LeetCode dataset ^[103]	Contains 1,825 Leetcode problems as of April 2021	Software Engineering Tasks	[93]
CUAD dataset ^[104]	13,000+ labels in 510 commercial legal contracts for clause identification under legal supervision	Requirements Elicitation	[40]
CQ2SPARQLOWL Dataset ^[105]	Schema-level competency questions and their corresponding SPARQL-OWL patterns	Requirements Modelling	[67]
Requirements datasets (user stories) ^[106]	22 datasets of 50+ requirements each, expressed as user stories from public or disclosed company sources	Requirements Modelling	[68]

Table 13: Overview of publicly available datasets used in primary studies.

5.4.2. RQ4.2: Source Code and Prompts

In total, 36 out of 74 primary studies (48%) provided one or both artefacts, while 38 papers (52%) did not provide supplementary materials related to their work. A comprehensive list of available source code and prompts used in these studies is provided in Appendix A, specifically in Table A.18. While this indicates a growing awareness of open science, more work is still needed to ensure replicability and verifiability of results.

¹³<https://zenodo.org/records/10959496>

¹⁴<https://zenodo.org/records/11000349>

¹⁵<https://zenodo.org/records/1414117>

¹⁶<https://data.mendeley.com/datasets/4ysx9fyzv4/1>

¹⁷<https://www.kaggle.com/datasets/onurgunes1993/rico-dataset>

¹⁸<https://www.kaggle.com/datasets/gzipchrist/leetcode-problem-dataset>

¹⁹<https://www.atticusprojectai.org/cuad>

²⁰<https://github.com/CQ2SPARQLOWL/Dataset>

²¹<https://github.com/RELabUU/revv-light>

5.4.3. RQ4.3: Large Language Models

We now explore the landscape of LLMs adopted in the studies considering the LLM types and the number of parameters. Since many studies employed different versions of LLMs from the same family (e.g., GPT-3 and GPT-4), we grouped the models according to their broader family, instead of reporting each specific model instance. In general, we identified five major families: *GPT*, *LLaMA*, *Mixtral/Mistral*, *CodeLLaMA*, and *Other*; we also included *BERT* as a baseline model family, as it was often used for comparison with newer LLMs—we recall that in this paper we do not classify BERT among the LLMs. Table 14 provides an overview of the LLM families used across the primary studies. The *GPT* family is by far the most prevalent, cited in 57 of 74 studies (77%). This is followed by *LLaMA* (9 studies, 12%), *BERT* (8 studies, 11%), and smaller families such as *Mixtral/Mistral* and *CodeLLaMA* (each in 3 studies, 4%). The *Other* category comprises 19 studies (26%) and reflects the wide diversity of emerging or specialized models (e.g., Codex, GPT-J, Falcon) used in research. As several studies employed more than one model family, some references appear in multiple rows.

Across the primary studies, the number of distinct LLM models (regardless of family) referenced per study ranged from a minimum of 1 to a maximum of 6. The median number of distinct LLMs cited per study was 1, indicating that most studies focused on a single model. Although this is in line with the exploratory nature of the studies, it should not be considered a recommended practice, as the obsolescence of models can quickly limit the validity and reproducibility of results. Evaluating multiple LLMs not only mitigates this risk but also provides a more robust understanding of how findings generalize across architectures, model families, and release cycles. On the other hand, it is worth noting that a non-negligible number of researchers are sensitive to the problem, as 29 studies (39.19%) utilized more than one LLM. Among these: 12 studies (16.22%) referenced two models; 8 studies (10.81%) cited three models; 5 studies (6.76%) used four models; 3 studies (4.05%) included five models; and 1 study (1.35%) employed as many as 6 models.

Using LLMs can have high costs if models are accessed as services, or require specialised infrastructures, and this can discourage the comparison of multiple LLMs. However, one can use strategies to mitigate this by making targeted comparisons: selecting representative models from different families, focusing on models of varying sizes or capabilities, leveraging open-source

alternatives when possible, or referring to benchmark leaderboards²². Such approaches allow researchers to gain meaningful insights while controlling costs and resource requirements, without sacrificing the robustness of the study.

Table 14: LLM’s family usage across primary studies.

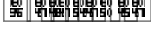
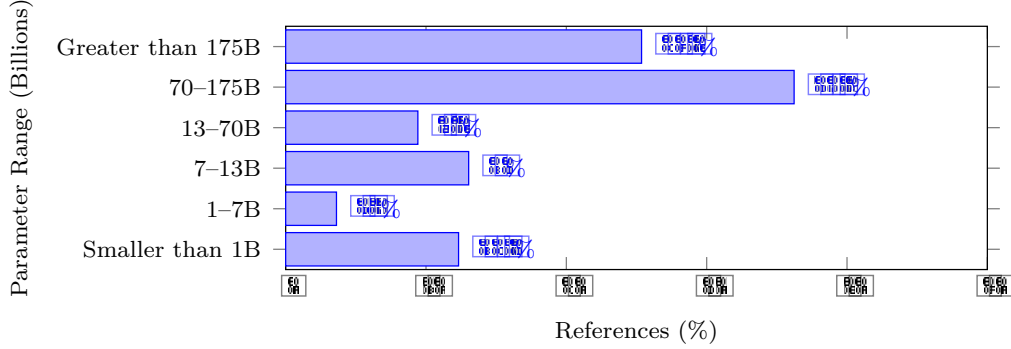
			
GPT	[65, 55, 95, 52, 89, 31, 81, 33, 46, 101, 47, 73, 77, 93, 67, 72, 37, 45, 58, 100, 40, 69, 61, 97, 50, 51, 43, 66, 35, 36, 57, 82, 63, 48, 87, 88, 29, 78, 32, 34, 59, 94, 39, 79, 64, 41, 84, 75, 91, 92, 68, 49, 62, 42, 90, 98, 56]	57	77%
LLaMA	[65, 55, 74, 67, 45, 90, 54, 93, 96]	9	12%
BERT	[44, 30, 76, 70, 100, 95, 48, 54]	8	11%
Mixtral/Mistral	[100, 69, 90]	3	4%
CodeLLaMA	[99, 45, 64]	3	4%
Other	[76, 38, 44, 30, 71, 87, 73, 56, 85, 86, 99, 80, 61, 90, 97, 88, 83, 48, 102]	19	26%

Fig. 12 also shows the number of parameters of the models, which indicates their size. Interestingly, many studies used LLMs with a high number of parameters, which often implies reliance on external services, since most universities and smaller organizations lack the hardware resources to run models with over 70B parameters locally—these LLMs require specialized hardware (multiple high-end GPUs or TPUs) and have huge memory requirements. While feasible for research purposes, this approach is less realistic in practical industrial settings, where companies are unlikely to expose sensitive requirements data to third-party services such as OpenAI or Google. This highlights a gap between experimental studies and real-world adoption, and suggests a potential need for smaller, locally deployable models or privacy-preserving solutions for LLM4RE applications.

²²https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard

Figure 12: Model mentions by parameter range



5.5. RQ4.4: Tools and Solutions

Table 15 summarizes 21 tools and solutions identified across the literature that support a variety of RE/SE tasks, along with their corresponding references. Of these, 17 (80%) focus on a large variety of RE tasks, from *Requirements Elicitation* (5 studies, 23.8%), to *Requirements Modeling* (2 studies, 10.3%) and *Defect Detection* (3 studies, 17.1%), while 4 (20%) support *SE Tasks*, including code generation and testing. These artifacts collectively represent a relatively small subset of the broader pool of primary studies. This indicates that the studies normally consider LLMs out of the box, without combining them in a meaningful way in a more complex workflows. Since most RE tasks are ‘hairy’ [111], we argue that meaningful progress will require going beyond simple, out-of-the-box applications of LLMs. Future research should investigate how LLMs can be integrated into richer workflows—possibly in combination with other AI techniques, formal methods, or domain-specific tools—to handle the complexity and multi-stakeholder nature of RE tasks.

Figure 13: Distribution of solutions/tools across RE/SE tasks

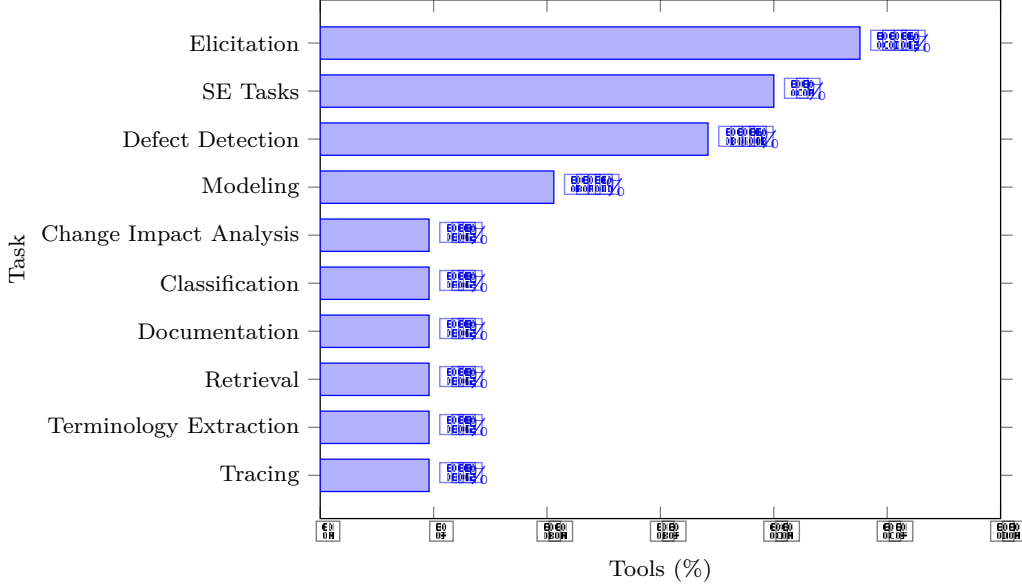


Table 15: Tools, supported RE/SE tasks, descriptions, and references

Tool	Supported RE/SE Tasks	Description	References
MURCIA	Change Impact Analysis	Uses language models to assist in analyzing regulatory changes, supporting human analysts.	[52]
ReqCompletion	Requirements Documentation	Recommends next token in real-time to enhance the requirements documentation process.	[44]
RAGTAG	SE Tasks	Generates test scenarios using Retrieval-Augmented Generation with LLMs, integrating domain knowledge.	[89]
GPT Agent	Requirements Elicitation	Supports eliciting requirements using a GPT-based agent.	[29]
IoT Tool	Requirements Modeling	Facilitates modeling of requirements for IoT systems.	[78]
CGS	Requirements Terminology Extraction	Extracts Contracts Grammar to capture contractual clause essence, identifying constituents.	[54]
Security Tool	Requirements Tracing	Supports tracing requirements for security-related tasks.	[77]
ClarifyGPT	SE Tasks	Enhances code generation by identifying ambiguous requirements and asking clarifying questions using LLMs.	[40]

Table 15 Tools, supported RE/SE tasks, descriptions, and references—Continued from previous page

Tool	Supported RE/SE Tasks	References
ALICE	Requirements Defect Detection: Detects contradictions in formal requirements using formal logic and LLMs.	[55]
Code Gen Tool	SE Tasks: Supports software engineering tasks focused on generating code.	[63]
Local GPT	Requirements Retrieval: Facilitates retrieval of requirements using a localized GPT model.	[80]
CRAFTER	Requirements Elicitation: Automates persona creation using GPT-3.5, recommending tailored personas.	[81]
Pipeline Tool	Requirements Classification: Supports classification of requirements through a pipeline approach.	[34]
nl2spec	Requirements Modeling: Derives formal specifications in temporal logics from natural language using LLMs.	[71]
Mini-BAR	Requirements Elicitation: Mines user reviews in English and French using LLMs, classifying and summarizing them.	[61]
Weaver	Requirements Elicitation: Generates knowledge bases and recommends concepts interactively for software testing.	[96]
A-CodGen	SE Tasks: Generates code with fewer errors using code repository information, reducing redundancy.	[98]
TESTPILOT	SE Tasks: Generates unit tests for JavaScript project APIs using LLMs, automating adaptive test generation.	[97]
Paska	Requirements Defect Detection: Detects quality issues in natural language requirements, offering improvement recommendations.	[83]
ACCA	Requirements Defect Detection: Evaluates AI-generated code correctness for security using symbolic execution against reference implementations.	[84]
PersonaGen	Requirements Elicitation: Generates persona templates from user feedback using GPT-4 and knowledge graphs for agile analysis.	[60]

RQ4: Resources

- Public datasets are scarce: only 10 were identified, used in 16% of studies, with most work relying on private datasets, limiting replicability and comparability.
- Source code and prompts are available in 36 out of 74 studies (48%), showing growing awareness of open science but leaving room for improvement in transparency.
- GPT-family models dominate (77%), followed by LLaMA (12%) and others, though almost 40% of studies used more than one model to improve robustness.
- Many studies rely on very large models (>70B parameters), which raises issues of cost, infrastructure, and privacy, and calls for smaller, locally deployable alternatives.
- Only 21 tools were identified, mostly “out-of-the-box” uses of LLMs; future work should focus on integrating LLMs into richer workflows combining AI, formal methods, and domain-specific tools.

5.6. RQ5: Evaluation

We now look into the evaluation performed in the studies. In particular, we first look at how the output produced by the LLM is assessed, whether with quantitative methods, e.g., measuring precision and recall with respect to a ground truth, with qualitative one, e.g., performing thematic analysis of the output after this is generated, or with a mix of both approaches. Then, we look into the empirical evaluation strategy adopted in the study, for example, if a laboratory experiment was conducted, e.g., comparing LLMs, or if the paper reports an experimental simulation, i.e., a systematic evaluation in a less controlled, yet artificial, setting.

5.6.1. RQ5.1: LLM Output Evaluation

Table 16 summarizes the methods employed across primary studies for assessing LLM outputs. The methods are categorized into three types: *Quantitative Methods* (30 studies, 42%), which use numerical indicators for objective assessment and *Qualitative Methods* (12 studies, 16%), which rely

on subjective evaluations such as expert reviews to assess, e.g., contextual appropriateness, completeness, or correctness of the output; *Mixed Methods* (30 studies, 42%), which combine quantitative metrics and qualitative judgments to provide an evaluation from the two different perspectives. The distribution of evaluation methods for LLMs output across primary studies is also visualised in Fig. 14. *Quantitative Methods* dominate together with *Mixed Methods*. This suggests that LLM output cannot be evaluated only in terms of objective performance measures, but requires in-depth analysis. Indeed, LLMs are not only used to perform analytic tasks, e.g., *Requirements Classification* or *Defect Detection*, but are widely used for purely generative tasks, such as *Model Generation*, or *Requirements Elicitation*. For these tasks, it is often hard to define a ground truth “a priori” to apply quantitative measures—different generated models can satisfy the same requirements, and different requirements formulations produced by an LLM can be equally adequate. Furthermore, given the richness of the output produced by an LLM, a thorough inspection is often required—possibly involving multiple judges to mitigate subjectivity—to assess, e.g., the quality of the models, or of the LLM-elicited requirements.

Table 16: Evaluation methods for LLM output: definitions, references, and frequencies

Method	Definition	Frequency	Percentage
Mixed Methods	Combines quantitative metrics and qualitative assessments to evaluate LLM output, balancing statistical performance with human judgment to have numerical, objective indicators as well as in-depth insights. [52, 44, 89, 29, 74, 100, 90, 30, 91, 102, 53, 64, 93, 67, 68, 79, 99, 32, 69, 70, 45, 88, 101, 71, 47, 38, 62, 96, 43, 46]	30	42%
Quantitative Methods	Employs numerical measures, such as accuracy, BLEU scores, or perplexity, to objectively assess the performance and reliability of LLM-generated text. [48, 65, 75, 78, 31, 76, 54, 77, 92, 40, 63, 80, 33, 49, 82, 59, 94, 95, 83, 84, 50, 60, 61, 37, 73, 87, 85, 57, 58, 56]	30	42%
Qualitative Methods	Relies on subjective analysis, such as expert reviews, to evaluate the quality, creativity, and contextual appropriateness of LLM output. [66, 55, 81, 34, 72, 39, 35, 41, 36, 86, 42, 97]	12	16%

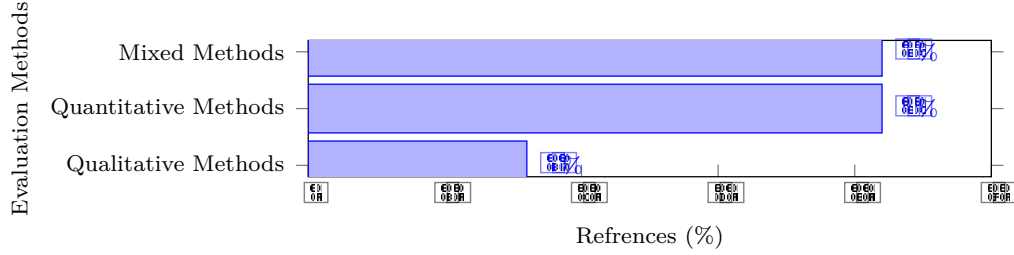


Figure 14: Histogram of evaluation methods for LLM output and corresponding study percentages

5.6.2. RQ5.2: Empirical Strategy

Table 17 summarizes the empirical strategies adopted across the primary studies, considering the classification by Stol and Fitzgerald [17]. The most frequently used strategy is *Laboratory Experiments*, reported in 56 studies (76%), which typically involve a systematic evaluation of the performance of LLMs against predefined ground truths. *Judgment Studies*, which rely on qualitative assessments of LLM output by a small group of individuals, account for 10 studies (14%). *Field Studies*, based on observations and interviews conducted in real-world contexts without introducing interventions, appear in 5 studies (7%). *Experimental Simulations*, which attempt to replicate real-world conditions without fully controlling all the variables, are used only in 3 studies (4%). *Sample Studies* and *Field Experiments* were not employed in any of the reviewed publications.

The dominance of *Laboratory Experiments* reflects the focus of the literature on controlled, repeatable evaluations of LLM performance, without considering real-world confounding factors. This is confirmed by the limited use of *Field Studies* and *Experimental Simulations*, which suggests a gap in understanding real-world applicability. The phenomenon is exacerbated by the absence of *Sample Studies* indicates that we currently have no evidence of the viewpoint of practitioners across different companies on LLM4RE, while the absence of *Field Experiments* indicates that, while observations in specific industries have been performed (with *Field Studies*), systematic evaluations are lacking. Overall, the current results suggest the pressing need to go into the field and gather evidence as well as opinions about LLM utility and effectiveness.

Table 17: Evaluation methods used across studies: definitions, references, and statistics

Method	Definition	Count	Percentage
Laboratory Experiments	Rigorous evaluation of LLM against a ground truth, typically developed by authors or public datasets, without company involvement. [52, 48, 44, 65, 100, 75, 91, 78, 102, 76, 64, 77, 92, 40, 67, 68, 79, 55, 99, 63, 80, 32, 33, 69, 49, 82, 70, 59, 72, 88, 101, 94, 39, 46, 58, 45, 95, 35, 36, 71, 47, 38, 61, 73, 41, 98, 97, 83, 84, 50, 51, 85, 86, 60, 56, 42]	56	76%
Experimental Simulation	Evaluation of LLM in a scenario mimicking real-world conditions, possibly compared with human subject behavior, without full control of all variables. [90, 31, 96]	3	3%
Judgment Study	Manual analysis of LLM output by 2–3 individuals providing qualitative opinions. [29, 74, 30, 66, 93, 81, 87, 57, 62, 37]	10	14%
Sample Study	Survey-based evaluation involving many subjects, yielding quantitative results, or analysis of multiple opinions through questionnaires. —	0	0%
Field Experiment	Study conducted in a real-world environment where a change is introduced, often involving company data or participants. —	0	0%
Field Study	Observational study in a real-world environment involving interviews or observations without introducing a specific tool or intervention. [89, 53, 54, 34, 43]	5	7%

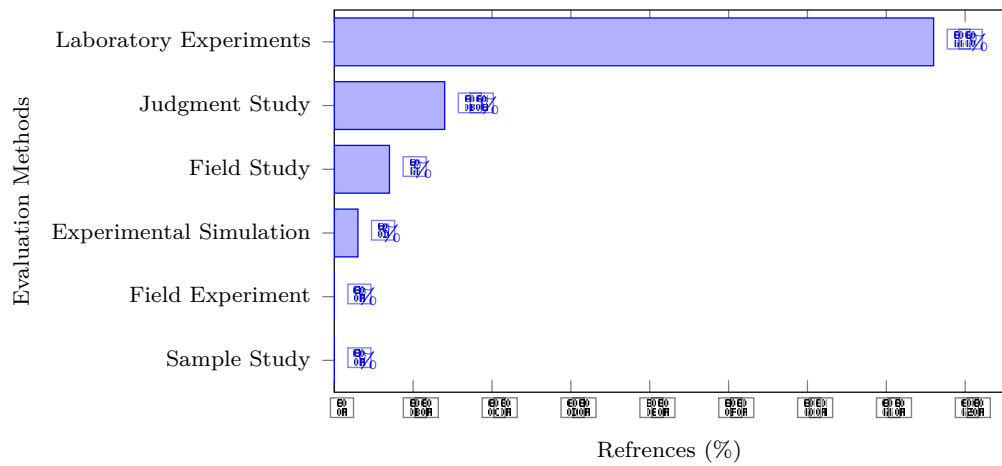


Figure 15: Histogram of evaluation methods and corresponding study percentages

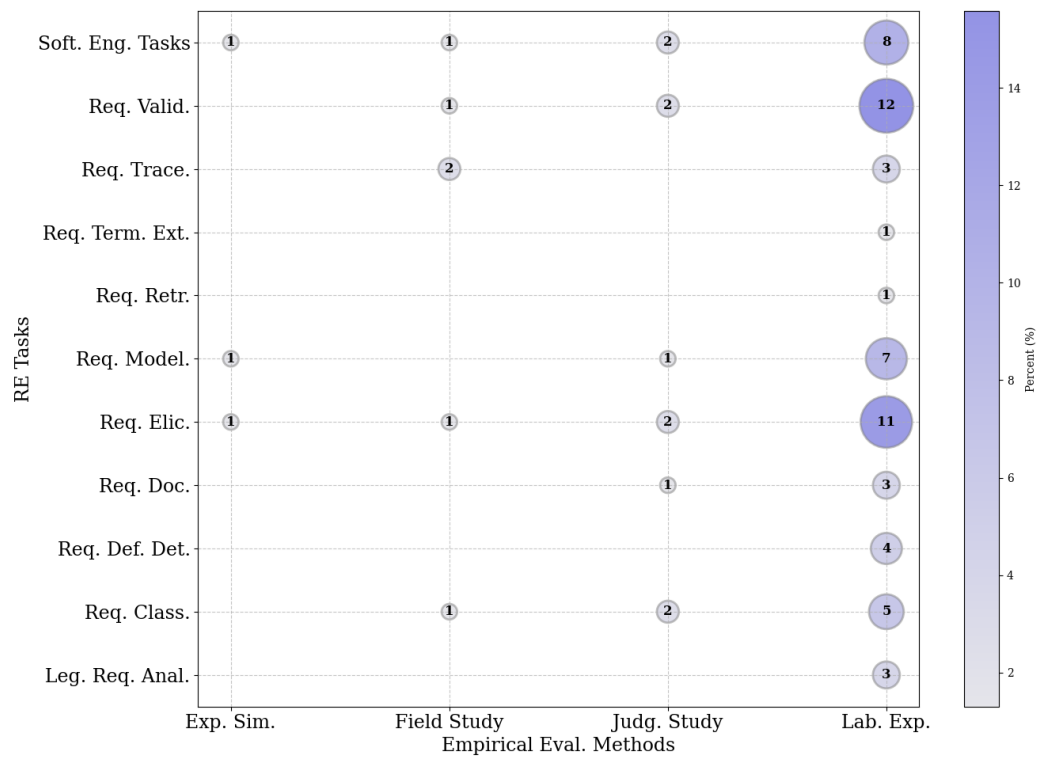


Figure 16: bubble plot of empirical strategy vs RE task

Figure 16 analyses empirical strategies across various RE tasks. *Laboratory Experiments* have been applied to all tasks, especially in *Requirements Validation* (13.79%), *Requirements Elicitation* (14.94%), *Software Engineering Tasks* (12.66%), and *Requirements Modelling* (8.05%). This diffusion highlights the flexibility of the strategy and its general suitability for all types of RE tasks. While no specific trends can be observed for the other strategies, due to their limited usage, we can observe other interesting outcomes when looking at the RE tasks: *Software Engineering Tasks* and *Requirements Elicitation* have been analysed with all types of empirical strategies. This suggests that the literature can provide evidence of different analysis perspectives for these tasks, considering both contrived settings [17] (e.g., *Laboratory Experiments* and *Judgment Studies*), and less artificial ones (e.g., *Experimental Simulations* and *Field Studies*). More research adopting the different strategies is instead required for the other tasks, to demonstrate the value of LLM4RE through different empirical lenses.

RQ5: Evaluation

- LLM output evaluation relies on three main approaches: *Quantitative* (42%), *Qualitative* (16%), and *Mixed Methods* (42%), reflecting the need to combine objective metrics with expert judgment, especially for those generative tasks where the output is rich, multiple correct outputs might exist, and a ground truth cannot be defined a priori (e.g., *Modeling*, *Elicitation*).
- *Laboratory Experiments* dominate empirical strategies (76%), providing controlled and repeatable performance evaluations but limited insight into real-world applicability.
- Other strategies—*Judgment Studies* (14%), *Field Studies* (7%), and *Experimental Simulations* (4%)—are underused, indicating a gap in evidence from real-world contexts and practitioner perspectives.
- Certain tasks, like *Elicitation* and *SE Tasks*, have been evaluated with all strategies, suggesting evidence on LLM effectiveness from different perspectives, whereas other RE tasks need more diverse empirical assessment.

6. Discussion

We now present and discuss key findings from our study, identify gaps in the existing literature, and suggest directions for future research.

6.1. LLM4RE as a Paradigm Shift

LLMs for RE represent a new and rapidly emerging research area, distinct from the earlier NLP4RE landscape. Unlike NLP4RE, which developed gradually over decades (1983–2019) and largely focused on analytic tasks such as defect detection and classification, LLM4RE began only in 2023, immediately following the release of ChatGPT. In just two years, 74 studies have been published, and publication counts rose 136% in 2024 compared to 2023, suggesting that LLMs quickly captured the attention of the RE community once easy-to-use interfaces lowered technical barriers. This rapid adoption contrasts with NLP4RE, where progress was slower and heavily concentrated in specialized venues.

In terms of tasks and artifacts, LLM4RE expands beyond the analytic focus of NLP4RE. While historical studies emphasized *Requirements Defect Detection* and *Classification*, LLMs are applied to cognitively demanding and generative tasks, such as *Requirements Elicitation*, *Validation*, and *Modeling*, often leveraging diverse input sources—from structured *Requirements Specifications* to informal artifacts like *User Feedback* and *Issues*—and producing a broader variety of outputs. This shift demonstrates the versatility of LLMs and their capacity to tackle tasks previously considered difficult or infeasible for traditional NLP or ML approaches.

Taken together, these factors—rapid growth, broader task coverage, and generative capabilities—suggest that LLM4RE is not merely an extension of NLP4RE, but represents a qualitatively new phase in the application of natural language analysis techniques in RE.

6.2. LLM4RE Expands the Scope of RE

We noted that 20% of the studies do not strictly focus on RE tasks, but on tasks that bridge RE and SE, such as test or code generation from requirements (collectively identified as *SE tasks*). This suggests a promising opportunity to re-center SE around requirements, and broaden the impact of the discipline. In many ways, RE becomes the foundation for other SE activities—such as coding, refactoring, testing, and maintenance—which can

increasingly be automated starting from high-quality requirements. Expertise in crafting clear, complete, and unambiguous requirements is therefore crucial, not only for traditional RE tasks but also for guiding automation across the software lifecycle. In latest years, topics such as testing and mining software repositories have dominated high-profile SE venues like ICSE and journals such as TSE, while RE research has been largely absent. With the advent of LLMs, RE researchers now have a chance to re-enter these venues, demonstrating the central role of requirements in shaping RE practices.

The distribution of venues of LLM4RE studies highlights cross-disciplinary interest which goes beyond SE: whereas NLP4RE work was largely confined to RE conferences, over 57% of LLM4RE studies appear in diverse venues including AI, NLP, and HCI forums (e.g., HCII, LREC-COLING, MSR, CAV). This reflects the opportunity for RE researchers to engage in cross-fertilization with other domains. Building on this cross-domain potential, the field’s expertise in requirement quality can directly inform practices in prompt engineering for LLMs. Just as requirements need to be precise and interpretable to ensure correct software behavior, prompts must be carefully formulated to obtain reliable model outputs [112]. In this sense, prompt engineering can be seen as a form of RE: the user specifies their needs in natural language to produce a desired result, i.e., it provides goals to be satisfied by an LLM or agent (i.e., the machine). Consequently, RE research has the potential to expand its impact beyond its traditional community, contributing to broader AI practices, especially those where the reliability of the output is more critical, such as healthcare or other mission-critical fields.

6.3. Rethinking LLM Applications in RE Tasks

Despite the extended scope of tasks and artifacts considered in LLM4RE, there is still considerable room for improvement. While *Classification*, *Defect Detection*, and *Modeling* tasks are commonly supported, the broader spectrum of tasks, such as *Terminology Extraction*, *Relations Extraction*, and *Change Impact Analysis*, remains underrepresented. It suggests that LLMs are not yet fully leveraged for activities that require contextual reasoning and domain-specific interpretation. Furthermore, despite their increased variety, input artifacts are still strongly dominated by structured documents like *Requirements Specifications* (38%), while dynamic or informal sources such as *User Feedback* (12%) or *Legal Texts* (9%), which are increasingly

critical in modern RE practice [113], are still underrepresented. Output artifacts also tend to mirror traditional deliverables, with few examples of studies generating *Traceability Links*, *Test Artifacts*, or *Assurance Cases*. The co-occurrence of inputs/outputs highlights a lack of diversity in transformation workflows. While *Requirements Specifications* are often used to generate other *Specifications* (typically refined/improved), *Models*, or *Code*, more exploratory combinations like *User Stories* to *Test Cases*, or *Elicitation Transcripts* to *Interview Scripts* appear in only 1% of studies each. This points to substantial opportunities for future research to expand the application of LLMs to underexplored LLM4RE tasks and input-output transformations. LLM-based approaches could greatly improve RE by supporting end-to-end workflows, especially in agile or user-centered contexts. For example, LLMs could analyze stakeholder chat transcripts to generate user stories [114], extract key terms from domain glossaries to ensure consistent terminology, or identify compliance risks by interpreting legal documents. These capabilities can help teams automate documentation, improve requirement quality, and maintain alignment with regulatory and user needs across the entire RE process.

Another important aspect that should be considered is leveraging the generative capabilities of LLM to complement analytic tasks with *explanations*. For example, tasks such as *Requirements Classification* or *Defect Detection* often suffer from ambiguous ground truths, as different annotators frequently interpret and label the same artifacts differently [115]. In this context, LLMs can be leveraged to provide justificatory explanations for their outputs: even when the underlying model is not inherently interpretable, it can generate convincing arguments that support and rationalize its predictions. In this sense, LLMs can not only enhance analytic tasks but also complement and improve generative and interpretative aspects of RE. Initial contributions in this direction [116] have demonstrated their potential to provide justificatory explanations and increase acceptance by users.

6.4. Toward Smarter Prompts for Requirements Engineering

Prompt engineering in LLM-based RE approaches remains highly experimental and heterogeneous. While strategies such as *Zero-shot* and *Few-shot* prompting dominate current practice, a substantial number of studies also leverage more structured techniques like task decomposition, template-based prompting, or context-rich designs. However, over 62% of the studies combine multiple strategies, often in ad hoc ways, suggesting a lack of

availability or/and knowledge about systematic frameworks for prompt composition. The limited use of interactive or RAG-based methods highlights a missed opportunity to leverage LLMs’ full reasoning and retrieval capabilities for complex RE tasks such as stakeholder dialogue, traceability, or requirements validation. Prompt selection is often underreported or based on informal trial-and-error, and only a small fraction of studies explicitly reference established prompt engineering guidelines. This signals a maturity gap in methodological rigor within RE research leveraging LLMs. For practitioners, these results imply that while LLMs offer flexibility, consistent outcomes depend heavily on prompt craft quality, underscoring the need for RE-specific prompt tooling, reuse patterns, and design standards. Future work should explore the development of systematic frameworks for prompt design and evaluation, with a focus on supporting the diverse range of tasks in RE. Automating prompt construction and selection according to task type could improve consistency and scalability. In this sense, the recent study by Zadenoori et al. [117] introduced a novel approach for iteratively optimizing prompts for requirements classification. This shows that there are many new creative approaches that can be studied in this field, which have not yet been examined or explored. We also recommend investigating how domain-specific ontologies and design rationale can be embedded into prompts to enhance the relevance and accuracy of LLM outputs. In addition, the RE community would benefit from empirical benchmarks that compare prompt strategies across representative RE scenarios. These benchmarks would help establish evidence-based best practices and support more informed decisions in prompt engineering.

6.5. Openness and Reproducibility

Openness in LLM4RE research remains insufficient. Only 12 out of 74 studies (16%) shared publicly available datasets. Although 10 datasets were identified, they were rarely reused, which limits opportunities for benchmarking and cross-study comparison. In contrast, 45 studies (61%) shared code or prompts, but 39% provided no supplementary materials at all. This lack of availability restricts reproducibility and slows progress in validating results, comparing methods, and building on previous work. Without access to data, code, or prompts, researchers cannot replicate experiments, assess the strength of evidence, or evaluate how well findings generalize to other contexts. The absence of transparency also hampers the development of standardized evaluation practices across the field. Making materials such as

datasets, source code, and prompts publicly available, and ensuring they are properly documented, would significantly enhance reproducibility. It would also enable meaningful cross-study validation of existing solutions. To support progress, the RE/SE community should promote shared repositories, adopt artefact availability standards, and encourage reproducible workflows. First steps in this direction have been done by the NLP4RE community itself [110], which proposed an ID card to be associated with empirical studies to facilitate replicability. We encourage adaptation and adoption of the card also in the LLM4RE field. A framework for performing empirical studies with LLMs for SE has also been recently published [118], with a strong focus on replicability and verifiability, giving further indications for authors to make their research truly open.

6.6. Model Usage and Practical Deployment

The dominance of the GPT family, cited in 77% of the reviewed studies, reflects a strong reliance on proprietary models. Although accessible via APIs, these models raise concerns about cost, control, and reproducibility. High usage costs make it unlikely that peers will reproduce results solely to verify findings. Surprisingly, only a minority of studies explored open-source alternatives such as LLaMA or Mixtral, suggesting promising directions for future research. The common reliance on a single model per study may reflect limited resources or a lack of awareness of comparative strengths. Future work benchmarking both commercial and open-source models could yield valuable insights into their suitability for RE tasks.

In parallel with model selection, some studies proposed practical LLM-based tools to support RE/SE tasks (21 tools in total). This is still a minority of the studies, suggesting that most of the researchers relied on LLMs “out-of-the-box”, rather than adapting and integrating them in appropriate workflows, which could provide more robust, context-aware, and multi-step support for complex RE tasks. The predominance of “out-of-the-box” applications confirms that the field is still largely exploratory: researchers are testing the capabilities of LLMs on isolated tasks rather than embedding them into end-to-end processes that reflect the multi-stakeholder, iterative, and interdependent nature of RE. Future work should focus on developing integrated LLM-based workflows, combining LLMs with other AI techniques, formal methods, and domain-specific tools, to handle challenges such as traceability, validation, and requirements evolution more effectively. This

shift would move LLM4RE from proof-of-concept studies toward practical, industrially relevant solutions.

6.7. Evaluation in Practice and Novel Frameworks

Our findings reveal that most studies (76%) rely on *Laboratory Experiments*, with limited use of methods contextualised in real-world settings like *Field Studies* (7%). While controlled settings offer consistency, they often overlook the complexity and unpredictability of actual RE practice. More *Sample Studies* (e.g., surveys in industry) need to be performed to provide evidence of the actual usage of LLM in RE in industry and the main tasks addressed, also to better target LLM4RE research towards goals that are considered relevant by practitioners. A first contribution in this sense has been given by Ullrich et al. [119], who interviewed 18 practitioners on their use of requirements artefacts for code generation. More *Field Experiments* are also needed, to avoid relying solely on practitioners' opinions, and provide evidence of LLM effectiveness in real-world scenarios.

The tendency to use *Mixed Methods*, besides *Quantitative* ones, to evaluate the output of LLMs is an important shift that acknowledges the complexity of LLM output, and the need to complement numerical evaluation with in-depth qualitative ones. However, evaluation should also go beyond checking the output of LLMs, but should consider the actual utility in practice. This implies designing evaluation frameworks that measure not only correctness or similarity to a reference but also the practical impact of LLMs on RE workflows, such as time savings, error reduction, or improved stakeholder communication. By incorporating metrics for real-world *effectiveness* and *usability*, researchers can better assess whether LLM-based solutions deliver tangible benefits beyond lab-controlled tasks. Moreover, combining *Mixed Methods* with *Field Studies* and *Experimental Simulations*, also considering longitudinal observations, can provide a more holistic understanding of LLMs' role in supporting both analytic and generative RE activities. Evaluation frameworks for LLMs such as the one proposed by Baltes *et al.* [118] should be appropriately extended to include evaluation of usefulness and acceptance (e.g., with the TAM/MEM model [120]), as well as other dimensions that are relevant for practice, such as the potential to reduce the cognitive workload of the users (e.g., with the NASA-TLX test [121]), while maintaining effectiveness.

7. Threats to Validity

In this section, we discuss the threats to validity of this study. We classified and mitigated them based on the guidelines by Apostolos *et al.* [122].

Study Selection Validity: A primary threat to validity includes the formulation of the search string and its likelihood of failure in returning all the relevant papers. The threat is further increased by the use of a single database, i.e., Scopus. To address these problems, we piloted the search string and also integrated part of the consolidated string from Zhao *et al.* [27], which was focused on NLP4RE. Furthermore, we included a secondary search, systematically analysing a wide range of relevant RE and SE venues. This approach aimed to improve coverage and capture any relevant papers that the initial query and search engine may have missed. A further threat is the subjectivity of inclusion and exclusion criteria. To mitigate the threat, we maximised the usage of objective indicators (e.g., Core ranking, number of pages).

Data Validity: The major threat to the validity of the data is the bias in data extraction due to subjectivity. This is partially mitigated by the cross-check performed by the last author—an expert in NLP4RE and LLM4RE—on a sample of the papers. This bias will be further mitigated in a future version of the paper, in which we will perform full cross-checking of the extracted data. While part of the classification schemes were derived from the literature, others were provided by the authors, which introduces subjectivity. However, the creation of the schemes was systematic and iterative, and involved two authors, which mitigates the threat.

Research Validity: Research validity is guaranteed by adhering to a defined protocol, which was defined in agreement between the first and last author. All results and analyses are shared openly to enable replication and independent review.

8. Summary and Conclusion

This paper presented a SLR of 74 primary studies published between 2023 and 2024, examining the landscape of LLM4RE. The survey has shown that LLM4RE represents not just a continuation of NLP4RE, but a paradigm shift in how natural language technologies are applied in requirements engineering. Unlike the gradual and narrowly scoped progress of NLP4RE, the adoption of LLMs has been both rapid and expansive, catalyzed by the availability of accessible interfaces and the versatility of generative models. Within just two

years, LLMs have been applied to a wide range of RE tasks, from elicitation and validation to modeling and tracing, and have begun to extend beyond RE into broader software engineering activities such as testing and code generation.

Our analysis highlights three major takeaways. First, LLM4RE re-centers requirements as a foundation for SE, offering new opportunities for RE to re-assert its central role in the discipline. Second, LLMs expand the methodological scope of RE, supporting generative and interpretative tasks, yet much potential remains untapped in underexplored areas such as terminology extraction, traceability, and compliance. Third, the success of LLM4RE depends on addressing critical gaps in openness, reproducibility, evaluation, and integration into practice. Currently, research is dominated by exploratory, proof-of-concept studies, often with limited methodological rigor, insufficient artifact sharing, and heavy reliance on proprietary models.

Looking forward, advancing LLM4RE requires both depth and breadth. Deeper progress will come from rigorous evaluation frameworks, reproducible practices, and systematic approaches to prompt engineering that recognize RE’s unique expertise in precision and clarity. Broader progress will come from expanding the scope of tasks, embracing diverse artifacts, and embedding LLMs into end-to-end RE workflows that reflect the realities of industrial practice.

In this sense, LLM4RE offers RE researchers a rare chance to reclaim centrality in SE, while simultaneously influencing the broader AI community—for example, by framing prompt engineering as a new form of requirements specification. By capitalizing on this momentum, the RE community can shape not only the future of SE, but also the reliability and trustworthiness of LLMs in safety-critical domains.

References

- [1] A. van Lamsweerde, Requirements engineering: a good practice guide, Springer Science & Business Media (2000).
- [2] I. Sommerville, Software engineering (10th ed.) (2016).
- [3] R. J. Abbott, D. Moorhead, Report on the workshop on requirements modeling, ACM SIGSOFT Software Engineering Notes 6 (3) (1981) 64–67.

- [4] A. Ferrari, A. Esuli, S. Gnesi, Identification of cross-domain ambiguity with language models, in: 2018 5th international workshop on artificial intelligence for requirements engineering (AIRE), IEEE, 2018, pp. 31–38.
- [5] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding (2019).

 URL 
- [6] K. Patel, H. Modi, U. Shah, Lace-hc: A lightweight attention-based classifier for efficient hierarchical classification of software requirements, in: Requirements Engineering: Foundation for Software Quality: 31st International Working Conference, REFSQ 2025, Barcelona, Spain, April 7–10, 2025, Proceedings, Springer-Verlag, Berlin, Heidelberg, 2025, p. 181–196.

 URL 
- [7] J. Lin, Y. Liu, Q. Zeng, M. Jiang, J. Cleland-Huang, Traceability transformed: Generating more accurate links with pre-trained bert models, in: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), IEEE, 2021, pp. 324–335.
- [8] OpenAI, Gpt-4 technical report (2023).
- [9] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., Llama: Open and efficient foundation language models, arXiv preprint arXiv:2302.13971 (2023).
- [10] DeepSeek-AI, D. Guo, D. Yang, ..., Z. Zhang, Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning (2025).

 URL 
- [11] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford,

- I. Sutskever, D. Amodei, Language models are few-shot learners, in: Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20, Curran Associates Inc., Red Hook, NY, USA, 2020.
- [12] A. See, A. Pappu, R. Saxena, A. Yerukola, C. D. Manning, Do massively pretrained language models make better storytellers? (2019).

 URL 
- [13] A. Zadenoori, aminzadenoori/llms-and-prompt-engineering-in-re—systematic-literature-review: Large language models (llms) for requirements engineering (re): A systematic mapping study (Sep. 2025).

 URL 
- [14] A. Ferrari, G. Ginde, Handbook on Natural Language Processing for Requirements Engineering, Springer, 2025.
- [15] T. B. Brown, et al., Language models are few-shot learners, Advances in Neural Information Processing Systems 33 (2020) 1877–1901.
- [16] A. Vaswani, et al., Attention is all you need, Advances in Neural Information Processing Systems 30 (2017) 5998–6008.
- [17] K.-J. Stol, B. Fitzgerald, The abc of software engineering research, ACM Transactions on Software Engineering and Methodology (TOSEM) 27 (3) (2018) 1–51.
- [18] G. Terry, N. Hayfield, V. Clarke, V. Braun, et al., Thematic analysis, The SAGE handbook of qualitative research in psychology 2 (17-37) (2017) 25.
- [19] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, J. M. Zhang, Large language models for software engineering: Survey and open problems, in: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), 2023, pp. 31–53. 
- [20] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, H. Wang, Large language models for software engineering:

A systematic literature review, ACM Transactions on Software Engineering and Methodology 33 (8), cited by: 13; All Open Access, Green Open Access (2024).

URL

- [21] M. Jin, S. Kumar, W. Zhang, Llms and llm-based agents in software engineering: A survey, Journal of Systems and Software 198 (2024) 111–130.
- [22] J. He, C. Treude, D. Lo, Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead, ACM Trans. Softw. Eng. Methodol. 34 (5) (May 2025).
- [23] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, Q. Wang, Software testing with large language models: Survey, landscape, and vision, IEEE Transactions on Software Engineering 50 (4) (2024) 911–936.
- [24] L. Schmid, T. Hey, M. Armbruster, S. Corallo, D. Fuchß, J. Keim, H. Liu, A. Koziolk, Software architecture meets llms: A systematic literature review (2025).
- [25] L. Zhao, W. Alhoshan, A. Ferrari, K. J. Letsholo, M. A. Ajagbe, E.-V. Chioasca, R. T. Batista-Navarro, Natural language processing for requirements engineering: A systematic mapping study, ACM Comput. Surv. 54 (3) (Apr. 2021).
- [26] B. Kitchenham, S. Charters, Guidelines for performing systematic literature reviews in software engineering, Engineering 45 (4) (2011) 1051–1057, technical Report EBSE 2007-001, Software Engineering Group, School of Computer Science and Mathematics, Keele University.
- [27] L. Zhao, W. Alhoshan, A. Ferrari, K. J. Letsholo, M. A. Ajagbe, E.-V. Chioasca, R. T. Batista-Navarro, Natural language processing for

requirements engineering: A systematic mapping study, *ACM Computing Surveys (CSUR)* 54 (3) (2021) 1–41.

- [28] A. Ferrari, Appendix B requirements engineering: Glossary, *Handbook on Natural Language Processing for Requirements Engineering* 507.
- [29] B. Görer, F. B. Aydemir, GPT-powered elicitation interview script generator for requirements engineering training, in: *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, Vol. 33, IEEE, 2024, pp. 372–379.
- [30] K. Devathasan, N. N. Arony, K. Gama, D. Damian, Deciphering empathy in developer responses: A hybrid approach utilizing the perception action model and automated classification, in: *2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW)*, IEEE, 2024, pp. 88–94.
- [31] A. Fantechi, S. Gnesi, L. Semini, Exploring LLMs’ ability to detect variability in requirements, in: *Lecture Notes in Computer Science, Lecture notes in computer science*, Springer Nature Switzerland, Cham, 2024, pp. 178–188.
- [32] K. Kolthoff, F. Kretzer, C. Bartelt, A. Maedche, S. P. Ponzetto, Interlinking user stories and GUI prototyping: A semi-automatic LLM-based approach, in: *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, Vol. 61, IEEE, 2024, pp. 380–388.
- [33] A.-R. Preda, C. Mayr-Dorn, A. Mashkoor, A. Egyed, Supporting High-Level to low-level requirements coverage reviewing with large language models, in: *Proceedings of the 21st International Conference on Mining Software Repositories*, ACM, New York, NY, USA, 2024, pp. 242–253.
- [34] A. Nouri, B. Cabrero-Daniel, F. Törner, H. Sivencrona, C. Berger, Engineering safety requirements for autonomous driving with large language models, in: *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, Vol. 33, IEEE, 2024, pp. 218–228.
- [35] T. Chang, S. Chen, G. Fan, Z. Feng, A self-iteration code generation method based on large language models, in: *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*, IEEE, 2023, pp. 275–281.

- [36] C. Treude, H. Hata, She elicits requirements and he tests: Software engineering gender bias in large language models, in: 2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR), IEEE, 2023.
- [37] K. Ronanki, C. Berger, J. Horkoff, Investigating ChatGPT’s potential to assist in requirements elicitation processes, in: 2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), IEEE, 2023, pp. 354–361.
- [38] C. Jain, P. R. Anish, A. Singh, S. Ghaisas, A transformer-based approach for abstractive summarization of requirements from obligations in software engineering contracts, in: 2023 IEEE 31st International Requirements Engineering Conference (RE), IEEE, 2023.
- [39] N. Nikolaidis, K. Flamos, K. Gulati, D. Feitosa, A. Ampatzoglou, A. Chatzigeorgiou, A comparison of the effectiveness of ChatGPT and co-pilot for generating quality python code solutions, in: 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion (SANER-C), Vol. 203, IEEE, 2024, pp. 93–101.
- [40] F. Mu, L. Shi, S. Wang, Z. Yu, B. Zhang, C. Wang, S. Liu, Q. Wang, ClarifyGPT: A framework for enhancing LLM-based code generation via requirements clarification, *Proc. ACM Softw. Eng.* 1 (FSE) (2024) 2332–2354.
- [41] B. Chen, K. Chen, S. Hassani, Y. Yang, D. Amyot, L. Lessard, G. Mussbacher, M. Sabetzadeh, D. Varró, On the use of GPT-4 for creating goal models: An exploratory study, in: 2023 IEEE 31st International Requirements Engineering Conference Workshops (REW), IEEE, 2023, pp. 262–271.
- [42] A. Marczak-Czajka, J. Cleland-Huang, Using ChatGPT to generate human-value user stories as inspirational triggers, in: 2023 IEEE 31st International Requirements Engineering Conference Workshops (REW), IEEE, 2023, pp. 52–61.
- [43] L. Xie, H. Xiao, Z. Li, Augmenting the problem frames approach with explicit data descriptions using ChatGPT, in: 2023 IEEE 31st International Requirements Engineering Conference Workshops (REW), IEEE, 2023, pp. 178–183.

- [44] X. Lian, J. Ma, H. Lv, L. Zhang, ReqCompletion: Domain-enhanced automatic completion for software requirements, in: 2024 IEEE 32nd International Requirements Engineering Conference (RE), Vol. abs/1611.08307, IEEE, 2024, pp. 142–154.
- [45] Y. Yang, Q. Zhang, C. Li, D. S. Marta, N. Batool, J. Folkesson, Human-centric autonomous systems with LLMs for user command reasoning, in: 2024 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW), IEEE, 2024, pp. 988–994.
- [46] Z. Li, D. Shin, Mutation-based consistency testing for evaluating the code understanding capability of LLMs, in: Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI, ACM, New York, NY, USA, 2024.
- [47] K. Ruan, X. Chen, Z. Jin, Requirements modeling aided by ChatGPT: An experience in embedded systems, in: 2023 IEEE 31st International Requirements Engineering Conference Workshops (REW), IEEE, 2023, pp. 170–177.
- [48] M. Fazelnia, V. Koscinski, S. Herzog, M. Mirakhorli, Lessons from the use of natural language inference (NLI) in requirements engineering tasks, in: 2024 IEEE 32nd International Requirements Engineering Conference (RE), Vol. 33, IEEE, 2024, pp. 103–115.
- [49] R. Lutze, K. Waldhör, Generating specifications from requirements documents for smart devices using large language models (LLMs), in: Lecture Notes in Computer Science, Lecture notes in computer science, Springer Nature Switzerland, Cham, 2024, pp. 94–108.
- [50] L. L. Silva, J. R. d. Silva, J. E. Montandon, M. Andrade, M. T. Valente, Detecting code smells using ChatGPT: Initial insights, in: Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ACM, New York, NY, USA, 2024, pp. 400–406.
- [51] A. Fantechi, S. Gnesi, L. Passaro, L. Semini, Inconsistency detection in natural language requirements using ChatGPT: A preliminary evaluation, in: 2023 IEEE 31st International Requirements Engineering Conference (RE), IEEE, 2023.

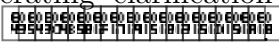
- [52] S. Abualhaija, M. Ceci, N. Sannier, D. Bianculli, L. C. Briand, D. Zetzsche, M. Bodellini, AI-enabled regulatory change analysis of legal requirements, in: 2024 IEEE 32nd International Requirements Engineering Conference (RE), Vol. 8, IEEE, 2024, pp. 5–17.
- [53] N. Feng, L. Marsso, S. G. Yaman, I. Standen, Y. Baatartogtokh, R. Ayad, V. O. de Mello, B. Townsend, H. Bartels, A. Cavalcanti, R. Calinescu, M. Chechik, Normative requirements operationalization with large language models, in: 2024 IEEE 32nd International Requirements Engineering Conference (RE), Vol. 09722, IEEE, 2024, pp. 129–141.
- [54] G. Rejithkumar, P. R. Anish, S. Ghaisas, Towards understanding contracts grammar: A large language model-based extractive question-answering approach, in: 2024 IEEE 32nd International Requirements Engineering Conference (RE), Vol. 21, IEEE, 2024, pp. 310–320.
- [55] A. E. Gärtner, D. Göhlich, Automated requirement contradiction detection through formal logic and LLMs, *Autom. Softw. Eng.* 31 (2) (Nov. 2024).
- [56] A. D. Rodriguez, K. R. Dearstyne, J. Cleland-Huang, Prompts matter: Insights and strategies for prompt engineering in automated software traceability, in: 2023 IEEE 31st International Requirements Engineering Conference Workshops (REW), IEEE, 2023, pp. 455–464.
- [57] M. Binder, V. Mezhuyev, A framework for creating an IoT system specification with ChatGPT, *Internet of Things* 27 (101218) (2024) 101218.
- [58] A. El-Hajjami, N. Fafin, C. Salinesi, Requirements-classifiers-evaluation-materials (2024).
- [59] J. J. Norheim, E. Rebentisch, Structuring natural language requirements with large language models, in: 2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW), IEEE, 2024, pp. 68–71.
- [60] X. Zhang, L. Liu, Y. Wang, X. Liu, H. Wang, A. Ren, C. Arora, PersonaGen: A tool for generating personas from user feedback, in:

2023 IEEE 31st International Requirements Engineering Conference (RE), IEEE, 2023.

- [61] J. Wei, A.-L. Courbis, T. Lambolais, B. Xu, P. L. Bernard, G. Dray, Zero-shot bilingual app reviews mining with large language models, in: 2023 IEEE 35th International Conference on Tools with Artificial Intelligence (ICTAI), IEEE, 2023, pp. 898–904.
- [62] A. Marczak-Czajka, J. Cleland-Huang, Using ChatGPT to generate human-value user stories as inspirational triggers, in: 2023 IEEE 31st International Requirements Engineering Conference Workshops (REW), IEEE, 2023, pp. 52–61.
- [63] B. Wei, Requirements are all you need: From requirements to code with LLMs, in: 2024 IEEE 32nd International Requirements Engineering Conference (RE), Vol. 35, IEEE, 2024, pp. 416–422.
- [64] M. Krishna, B. Gaur, A. Verma, P. Jalote, Using LLMs in software requirements specifications: An empirical evaluation, in: 2024 IEEE 32nd International Requirements Engineering Conference (RE), Vol. 1, IEEE, 2024, pp. 475–483.
- [65] N. Klievtsova, J. Mangler, T. Kampik, S. Rinderle-Ma, Utilizing process models in the requirements engineering process through Model2Text transformation, in: 2024 IEEE 32nd International Requirements Engineering Conference (RE), Vol. 1, IEEE, 2024, pp. 205–217.
- [66] A. Ferrari, S. Abualhaija, C. Arora, Model generation with LLMs: From requirements to UML sequence diagrams, in: 2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW), Vol. 24, IEEE, 2024, pp. 291–300.
- [67] R. Alharbi, V. Tamma, F. Grasso, T. Payne, An experiment in retrofitting competency questions for existing ontologies, in: Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing, ACM, New York, NY, USA, 2024.
- [68] M. Bragilovski, A. T. Van Can, F. Dalpiaz, A. Sturm, Deriving domain models from user stories: Human vs. machines, in: 2024 IEEE 32nd

International Requirements Engineering Conference (RE), Vol. 3378, IEEE, 2024, pp. 31–42.

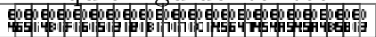
- [69] G. Rejithkumar, P. R. Anish, J. Shukla, S. Ghaisas, Probing with precision: Probing question generation for architectural information elicitation, in: Proceedings of the 1st IEEE/ACM Workshop on Multi-disciplinary, Open, and RElevant Requirements Engineering, Vol. 35, ACM, New York, NY, USA, 2024, pp. 8–14.
- [70] M. I. Azeem, S. Abualhaija, A multi-solution study on GDPR AI-enabled completeness checking of DPAs, *Empir. Softw. Eng.* 29 (4) (Jul. 2024).
- [71] M. Cosler, C. Hahn, D. Mendoza, F. Schmitt, C. Trippel, Nl2spec: Interactively translating unstructured natural language to temporal logics with large language models, in: Lecture Notes in Computer Science, Lecture notes in computer science, Springer Nature Switzerland, Cham, 2023, pp. 383–396.
- [72] J. Zhang, Y. Chen, C. Liu, N. Niu, Y. Wang, Empirical evaluation of ChatGPT on requirements information retrieval under zero-shot setting, in: 2023 International Conference on Intelligent Computing and Next Generation Networks(ICNGN), IEEE, 2023.
- [73] S. Arulmohan, M.-J. Meurs, S. Mosser, Extracting domain models from textual requirements in the era of large language models, in: 2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), IEEE, 2023.
- [74] S. Lubos, A. Felfernig, T. N. T. Tran, D. Garber, M. El Mansi, S. P. Erdeniz, V.-M. Le, Leveraging LLMs for the quality assurance of software requirements, in: 2024 IEEE 32nd International Requirements Engineering Conference (RE), Vol. 6, IEEE, 2024, pp. 389–397.
- [75] K. K. Shahandashti, A. B. Belle, M. M. Mohajer, O. Odu, T. C. Lethbridge, H. Hemmati, S. Wang, Using GPT-4 turbo to automatically identify defeaters in assurance cases, in: 2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW), Vol. 35, IEEE, 2024, pp. 46–56.

- [76] T. Hey, J. Keim, S. Corallo, Requirements classification for traceability link recovery, in: 2024 IEEE 32nd International Requirements Engineering Conference (RE), IEEE, 2024, pp. 155–167.
- [77] J. Hassine, An LLM-based approach to recover traceability links between security requirements and goal models, in: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, ACM, New York, NY, USA, 2024.
- [78] S. Chen, X. Chen, Z. Jin, Building device models using LLMs for requirements simulation in smart homes, in: 2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW), IEEE, 2024, pp. 176–183.
- [79] M. Sivakumar, A. B. Belle, J. Shan, K. Khakzad Shahandashti, Prompting GPT –4 to support automatic safety case generation, Expert Syst. Appl. 255 (124653) (2024) 124653.
- [80] Y. Uygun, V. Momodu, Local large language models to simplify requirement engineering documents in the automotive industry, Prod. Manuf. Res. 12 (1) (Dec. 2024).
- [81] D. Karolita, J. Grundy, T. Kanij, H. Obie, J. McIntosh, CRAFTER: A persona generation tool for requirements engineering, in: Proceedings of the 19th International Conference on Evaluation of Novel Approaches to Software Engineering, SCITEPRESS - Science and Technology Publications, 2024, pp. 674–683.
- [82] A. Singhal, C. Jain, P. R. Anish, A. Chakraborty, S. Ghaisas, Generating clarification questions for disambiguating contracts (2024).

- [83] A. Veizaga, S. Y. Shin, L. C. Briand, Automated smell detection and recommendation in natural language requirements, IEEE Trans. Softw. Eng. (2024) 1–26.
- [84] D. Cotroneo, A. Foggia, C. Improta, P. Liguori, R. Natella, Automating the correctness assessment of AI-generated code for security contexts, J. Syst. Softw. 216 (112113) (2024) 112113.

- [85] V. Bertram, H. Kausch, E. Kusmenko, H. Nqiri, B. Rumpe, C. Venhoff, Leveraging natural language processing for a consistency checking toolchain of automotive requirements, in: 2023 IEEE 31st International Requirements Engineering Conference (RE), IEEE, 2023.
- [86] E. Soares, M. Aranda, N. Oliveira, M. Ribeiro, R. Gheyi, E. Souza, I. Machado, A. Santos, B. Fonseca, R. Bonifácio, Manual tests do smell! cataloging and identifying natural language test smells, in: 2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), IEEE, 2023.
- [87] L. Görgen, E. Müller, M. Triller, B. Nast, K. Sandkuhl, Large language models in enterprise modeling: Case study and experiences, in: Proceedings of the 12th International Conference on Model-Based Software and Systems Engineering, SCITEPRESS - Science and Technology Publications, 2024.
- [88] C. Arora, M. Sabetzadeh, L. Briand, F. Zimmer, Extracting domain models from natural-language requirements, in: Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems, ACM, New York, NY, USA, 2016.
- [89] C. Arora, T. Herda, V. Homm, Generating test scenarios from NL requirements using retrieval-augmented LLMs: An industrial study, in: 2024 IEEE 32nd International Requirements Engineering Conference (RE), Vol. 33, IEEE, 2024, pp. 240–251.
- [90] H. Hasso, B. Fischer-Starcke, H. Geppert, Quest-RE QUestion generation and exploration STrategy for requirements engineering, in: 2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW), IEEE, 2024, pp. 1–9.
- [91] T. Yaacov, A. Elyasaf, G. Weiss, Boosting LLM-based software generation by aligning code with requirements, in: 2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW), Vol. 33, IEEE, 2024, pp. 301–305.
- [92] K. Khakzad Shahandashti, M. Sivakumar, M. M. Mohajer, A. Boaye Belle, S. Wang, T. Lethbridge, Assessing the impact of GPT-4 turbo in generating defeaters for assurance cases, in: Proceedings of

the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering, ACM, New York, NY, USA, 2024.

- [93] Y. Li, J. Keung, Z. Yang, X. Ma, J. Zhang, S. Liu, SimAC: simulating agile collaboration to generate acceptance criteria in user story elaboration, *Autom. Softw. Eng.* 31 (2) (Nov. 2024).
- [94] S. Ren, H. Nakagawa, T. Tsuchiya, Harnessing LLM conversations for goal model generation from user reviews, in: *Proceedings of the 16th International Conference on Agents and Artificial Intelligence*, SCITEPRESS - Science and Technology Publications, 2024.
- [95] A. Tikayat Ray, B. F. Cole, O. J. Pinon Fischer, A. P. Bhat, R. T. White, D. N. Mavris, Agile methodology for the standardization of engineering requirements using large language models, *Systems* 11 (7) (2023) 352.
- [96] C. Yang, R. Rustogi, R. Brower-Sinning, G. Lewis, C. Kaestner, T. Wu, Beyond testers' biases: Guiding model testing with knowledge bases using LLMs, in: *Findings of the Association for Computational Linguistics: EMNLP 2023*, Association for Computational Linguistics, Stroudsburg, PA, USA, 2023.
- [97] M. Schäfer, S. Nadi, A. Eghbali, F. Tip, An empirical evaluation of using large language models for automated unit test generation, *IEEE Trans. Softw. Eng.* 50 (1) (2024) 85–105.
- [98] D. Liao, S. Pan, X. Sun, X. Ren, Q. Huang, Z. Xing, H. Jin, Q. Li, A3-CodGen: A repository-level code generation framework for code reuse with local-aware, global-aware, and third-party-library-aware, *IEEE Trans. Softw. Eng.* 50 (12) (2024) 3369–3384.
- [99] M. North, A. Atapour-Abarghouei, N. Bencomo, Code gradients: Towards automated traceability of LLM-generated code, in: *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, Vol. 378, IEEE, 2024, pp. 321–329.
- [100] S. Hassani, M. Sabetzadeh, D. Amyot, J. Liao, Rethinking legal compliance automation: Opportunities with large language models, in: *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, Vol. 33, IEEE, 2024, pp. 432–440.

- [101] S. de Kinderen, K. Winter, Towards taming large language models with prompt templates for legal GRL modeling, in: Lecture Notes in Business Information Processing, Lecture notes in business information processing, Springer Nature Switzerland, Cham, 2024, pp. 213–228.
- [102] C. Arora, J. Grundy, L. Puli, N. Layton, Towards standards-compliant assistive technology product specifications via LLMs, in: 2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW), Vol. 33, IEEE, 2024, pp. 385–389.
- [103] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff, et al., The prompt report: a systematic survey of prompt engineering techniques, arXiv preprint arXiv:2406.06608 (2024).
- [104] J. He, M. Rungta, D. Koleczek, A. Sekhon, F. X. Wang, S. Hasan, Does prompt formatting have any impact on llm performance?, arXiv preprint arXiv:2411.10541 (2024).
- [105] I. Mohedas, S. R. Daly, R. P. Loweth, L. Huynh, G. L. Cravens, K. H. Sienko, The use of recommended interviewing practices by novice engineering designers to elicit information during requirements development, Design Science 8 (2022) e16. 
- [106] C. Arora, et al., Leveraging llms for automated requirements extraction, in: Proceedings of the International Conference on Software Engineering, 2023, pp. 1–10.
- [107] S. Ekin, Prompt engineering for chatgpt: A quick guide to techniques, tips, and best practices (May 2023). 

URL 
- [108] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. El-nashar, J. Spencer-Smith, D. C. Schmidt, A prompt pattern catalog to enhance prompt engineering with chatgpt (2023). 

URL 
- [109] M. Blaha, J. Rumbaugh, Object -Oriented Modeling and Design with UML: For VTU, 2/e, Pearson Education, 2005.
URL 

- [110] S. Abualhaija, F. B. Aydemir, F. Dalpiaz, D. Dell’Anna, A. Ferrari, X. Franch, D. Fucci, Replication in requirements engineering: the nlp for re case, *ACM Transactions on Software Engineering and Methodology* 33 (6) (2024) 1–33.
- [111] D. M. Berry, Empirical evaluation of tools for hairy requirements engineering tasks, *Empirical Software Engineering* 26 (6) (2021) 111.
- [112] G. Marvin, N. Hellen, D. Jjingo, J. Nakatumba-Nabende, Prompt engineering in large language models, in: *International conference on data intelligence and cognitive informatics*, Springer, 2023, pp. 387–402.
- [113] J. Dąbrowski, E. Letier, A. Perini, A. Susi, Analysing app reviews for software engineering: a systematic literature review, *Empirical Software Engineering* 27 (03 2022). 
- [114] M. Anders, B. Paech, Ferere: Feedback requirements relation using large language models, in: A. Hess, A. Susi (Eds.), *Requirements Engineering: Foundation for Software Quality*, Springer Nature Switzerland, Cham, 2025, pp. 89–105.
- [115] A. Ferrari, G. Gori, B. Rosadini, I. Trotta, S. Bacherini, A. Fantechi, S. Gnesi, Detecting requirements defects with nlp patterns: an industrial experience in the railway domain, *Empirical Software Engineering* 23 (6) (2018) 3684–3733.
- [116] S. Bashir, A. Ferrari, M. Abbas, P. E. Strandberg, Z. Haider, M. Saadatmand, M. Bohlin, Requirements ambiguity detection and explanation with llms: An industrial study, in: *International Conference on Software Maintenance and Evolution*, 2025.
- [117] M. A. Zadenoori, L. Zhao, W. Alhoshan, A. Ferrari, Automatic prompt engineering: The case of requirements classification, in: A. Hess, A. Susi (Eds.), *Requirements Engineering: Foundation for Software Quality*, Springer Nature Switzerland, Cham, 2025, pp. 217–225.
- [118] S. Baltes, F. Angermeir, C. Arora, M. M. Barón, C. Chen, L. Böhme, F. Calefato, N. Ernst, D. Falessi, B. Fitzgerald, et al., Evaluation guidelines for empirical studies in software engineering involving llms, *arXiv preprint arXiv:2508.15503* (2025).

- [119] J. Ullrich, M. Koch, A. Vogelsang, From requirements to code: Understanding developer practices in llm-assisted software engineering, arXiv preprint arXiv:2507.07548 (2025).
- [120] N. Marangunić, A. Granić, Technology acceptance model: a literature review from 1986 to 2013, Universal access in the information society 14 (1) (2015) 81–95.
- [121] S. Rubio, E. Díaz, J. Martín, J. M. Puente, Evaluation of subjective mental workload: A comparison of swat, nasa-tlx, and workload profile methods, Applied psychology 53 (1) (2004) 61–86.
- [122] A. Ampatzoglou, S. Bibi, P. Avgeriou, A. Chatzigeorgiou, Guidelines for Managing Threats to Validity of Secondary Studies in Software Engineering, Springer International Publishing, Cham, 2020, pp. 415–441.

URL

Appendix A. Code Availability

This appendix presents a table summarizing the code availability for RE tasks associated with specific research citations. The table includes three columns: **Citation**, **Req. Task**, and **Code**. The *Citation* corresponds to the reference identifier for each study. The *Req. Task* column lists the broad RE task category (e.g., Req. Elicitation, Software Engineering Tasks) derived from mapping specific tasks to a predefined set of categories based on semantic similarity. The *Code* column provides hyperlinks to repositories or datasets (e.g., GitHub, Zenodo, or DOI links) where the code or data is publicly accessible; only entries with available code are included. This table serves as a resource for researchers and practitioners seeking code implementations for RE-related tasks.

Table A.18: Requirements Engineering Tasks and Code Availability

Citation	Req. Task	Code
[52]	Req. Tracing; Legal Req. Analysis	Zenodo

Note: Full URLs are available in the original data source. Shortened URLs are used for brevity.

Citation	Req. Task	Code
[48]	Req. Classification; Req. Defect De- tection	Zenodo
[65]	Req. Modelling	GitHub
[89]	Software Engineering Tasks	Zenodo
[74]	Req. Validation	GitHub
[100]	Legal Req. Analysis	GitHub
[30]	Req. Elicitation	GitHub
[66]	Req. Modelling	GitHub
[102]	Software Engineering Tasks	Zenodo
[31]	Req. Elicitation	GitHub
[64]	Req. Terminology (Glossary) Extrac- tion	GitHub
[40]	Req. Elicitation	GitHub
[55]	Req. Tracing	ZLB
[99]	Software Engineering Tasks	GitHub
[63]	Req. Retrieval	DOI
[80]	Req. Validation	Zenodo
[32]	Req. Elicitation	Zenodo
[33]	Req. Elicitation	GitHub
[82]	Req. Validation	DOI
[70]	Req. Modelling	GitHub
[72]	Req. Modelling	DOI
[39]	Legal Req. Analysis	Zenodo
[87]	Req. Validation; Legal Req. Analysis	GitHub
[46]	Software Engineering Tasks	GitHub
[57]	Req. Elicitation	DOI
[58]	Req. Validation	GitHub
[45]	Req. Documentation	Mendeley
[95]	Req. Classification	GitHub

Note: Full URLs are available in the original data source. Shortened URLs are used for brevity.

Citation	Req. Task	Code
[98]	Software Engineering Tasks	GitHub
[97]	Software Engineering Tasks	GitHub
[83]	Req. Validation	Figshare
[84]	Software Engineering Tasks	GitHub
[50]	Req. Defect Detection	DOI
[51]	Req. Defect Detection	Zenodo
[60]	Req. Classification	GitHub

Note: Full URLs are available in the original data source. Shortened URLs are used for brevity.